CS52 · 5TH SEMESTER · CIE-1

Artificial Intelligence & Machine Learning

The complete CIE-1 study guide — from zero to exam-ready. Every topic in plain language, every class example solved step by step.

Unit I

What is AI · Intelligent agents · Problem solving by search (uninformed & informed) · Heuristics · Logical agents · Wumpus world · Propositional logic

Unit II

First-order logic · Inference in FOL (unification, forward/backward chaining, resolution) · Knowledge graphs & ontologies · Uncertainty & Bayes · Intro to ML · Find-S & Candidate-Elimination

Unit III (CIE-1 part)

Classification vs regression · Linear regression with examples · Naive Bayes classifier · Safety net: logistic regression, SVM, KNN

Course coordinator: Dr. Rajarajeswari S · Text books: Russell & Norvig, *AI: A Modern Approach* (4th ed.); Tom M. Mitchell, *Machine Learning* · CIE: 50 marks (avg. of two tests for 30 + lab 10 + case study 10)

0 How to use this guide

What's inside

This guide covers **everything for CIE-1**: all of Unit I, all of Unit II, and the three Unit III topics your CR shared (classification vs regression, Naive Bayes, linear regression with example). It merges your class notes, slides and the textbook, and solves every worked example from class step by step.

Each chapter follows the same pattern:

1. **Plain-English idea** first — what it is and why it exists.
2. **Formal definition** — the line the examiner expects in your answer.
3. **Diagram / table** — draw these in the exam.
4. **Worked example** — numbers checked.
5. **Likely questions + 60-second recap** at the end.

Box legend

DEFINITION
Write this word-for-word-ish.

KEY POINT
The idea that makes it click.

EXAM TIP
How to score full marks.

COMMON MISTAKE
Where students lose marks.

WORKED EXAMPLE
Solved problems.

MEMORY TRICK
Mnemonics.

★ Very likely = high chance in the exam
★ Likely = good chance
Yellow highlight = the key phrase to remember.

Study plan (fast track)

Session	What to study	Goal
Day 1	Ch 1–2 (AI, agents, PEAS, environment types, agent types) + Ch 3 search (BFS, DFS, UCS, IDS, A*)	Draw the 5 agent diagrams; trace A* on the Romania map
Day 2	Ch 4 logical agents & propositional logic + Ch 5 FOL	Truth table, CNF, resolution in PL; translate 10 English sentences to FOL
Day 3	Ch 6 inference in FOL + Ch 7 knowledge graphs	Unification table; crime example (FC + BC); "John is happy" by resolution
Day 4	Ch 8 uncertainty & Bayesian networks + Ch 9 intro to ML	Cancer MAP, PlayTennis NB, P(W) in a Bayes net; Find-S and Candidate-Elimination tables
Day 5	Ch 10–12 (Unit III) + Ch 13 skim	Comparison table; hours→marks regression; email NB with Laplace
Last night	Revision pages at the end: formula sheet, top-25 questions, checklist	Re-solve 1 numerical per topic without looking

HOW TO WRITE ANSWERS IN AN AIML PAPER

- **Definition → diagram → explanation → example → advantages/limitations.** Even a 5-mark answer should have a diagram or table.
- For numericals, write the **formula first**, then substitution, then the boxed answer with units/meaning.
- For proofs (chaining, resolution), number every clause and write the substitution on every step.
- Underline keywords: rational, admissible, entailment, unification, posterior, conditional independence.

ABOUT THE SYLLABUS

Two syllabus copies exist. The revised 2026 copy (from the class Drive) lists **First-Order Logic under Unit II** and marks **Uninformed Search** (Unit I) and **Knowledge Graphs & Ontologies** (Unit II) as self-study (SS) topics. Self-study topics are still examinable — they're fully covered here. For Unit III, CIE-1 covers only the three topics your CR shared; if the teacher announces more, Ch 13 gives you a head start.

Contents

Unit I — Agents, Search & Logic

1. What is AI?
2. Intelligent agents: PEAS, environments, agent types
3. Problem solving by search: BFS, UCS, DFS, DLS, IDS, bidirectional, greedy, A*, heuristics
4. Logical agents, Wumpus world, propositional logic, resolution, chaining

Unit II — Logic, Knowledge, Uncertainty, Intro to ML

5. First-order logic
6. Inference in FOL: UI/EI, unification, lifting, forward & backward chaining, resolution
7. Knowledge graphs & ontologies
8. Reasoning under uncertainty: Bayes, MAP, Bayesian networks, Naive Bayes
9. Introduction to ML: definitions, tasks, well-posed problems, types, Find-S, Candidate-Elimination

Unit III — CIE-1 portion

10. Classification vs regression
11. Linear regression (3 worked examples, R^2, polynomial)
12. Naive Bayes classifier (4 worked examples, Laplace)
13. Safety net: logistic regression, SVM, KNN

Revision

14. Formula sheet, top questions, last-night checklist

UNIT I

Intelligent Agents, Search & Logic

- **Ch 1 · What is AI?** four approaches, the Turing test, why we build rational agents
- **Ch 2 · Intelligent Agents** agents & environments, rationality, PEAS, environment types, the five agent structures
- **Ch 3 · Problem Solving by Searching** problem formulation, toy & real problems, search trees, uninformed search (BFS, UCS, DFS, DLS, IDS, bidirectional), informed search (greedy, A*), heuristic functions
- **Ch 4 · Logical Agents** knowledge-based agents, the Wumpus world, logic & entailment, propositional logic, equivalences, resolution, Horn clauses, forward & backward chaining, agents based on propositional logic

1 What is Artificial Intelligence?

Start here. This chapter gives you the vocabulary of the whole course: what "intelligent" means for a machine, the four ways people have defined AI, and why this textbook builds **rational agents**.

HOW TO STUDY UNIT I (PLAN)

- ☐ **Session 1:** Ch 1 + Ch 2 (definitions, PEAS, environment types, agent diagrams). These are easy marks.
- ☐ **Session 2:** Ch 3 up to uninformed search. Practise the traces on paper (frontier at every step).
- ☐ **Session 3:** Ch 3 informed search (A* on the Romania map, h_1/h_2).
- ☐ **Session 4:** Ch 4 (Wumpus world, truth table, equivalences, CNF, resolution, forward/backward chaining).
- ☐ **Before the test:** read every *60-second recap* box and attempt the *Likely exam questions*.

Tags: ★ Very likely = appears in almost every CIE/SEE paper; ★ Likely = asked often.

1.1 Intelligence and AI in plain words

Intelligence is the ability to *perceive* a situation, *understand / reason* about it, *decide*, and *act* so as to reach a goal — and to *learn* from experience. **Artificial Intelligence (AI)** is the field that tries to build machines (programs, robots) that show this kind of intelligent behaviour.

DEFINITION (WRITE THIS IN THE EXAM)

Artificial Intelligence is the study and design of intelligent agents: systems that perceive their environment through sensors and act upon it through actuators so as to achieve the best possible (expected) outcome. AI is concerned not only with *understanding* intelligence but also with *building* intelligent entities.

Different researchers defined AI differently. The textbook groups eight famous definitions along **two dimensions**:

- **Thinking vs Acting** — do we care about the *thought process / reasoning* inside, or only about the outside *behaviour*?
- **Human vs Rational** — do we measure success by *how closely it matches humans* (fidelity to human performance), or against an *ideal* standard of doing the right thing (**rationality**)?

Thinking Humanly	Thinking Rationally
"The exciting new effort to make computers think … *machines with minds*, in the full and literal sense." (Haugeland, 1985) "The automation of activities that we associate with human thinking — decision-making, problem solving, learning." (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak & McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly	**Acting Rationally**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things at which, at the moment, people are better." (Rich & Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole et al., 1998) "AI … is concerned with intelligent behaviour in artifacts." (Nilsson, 1998)

1.2 The four approaches to AI ★ Very likely

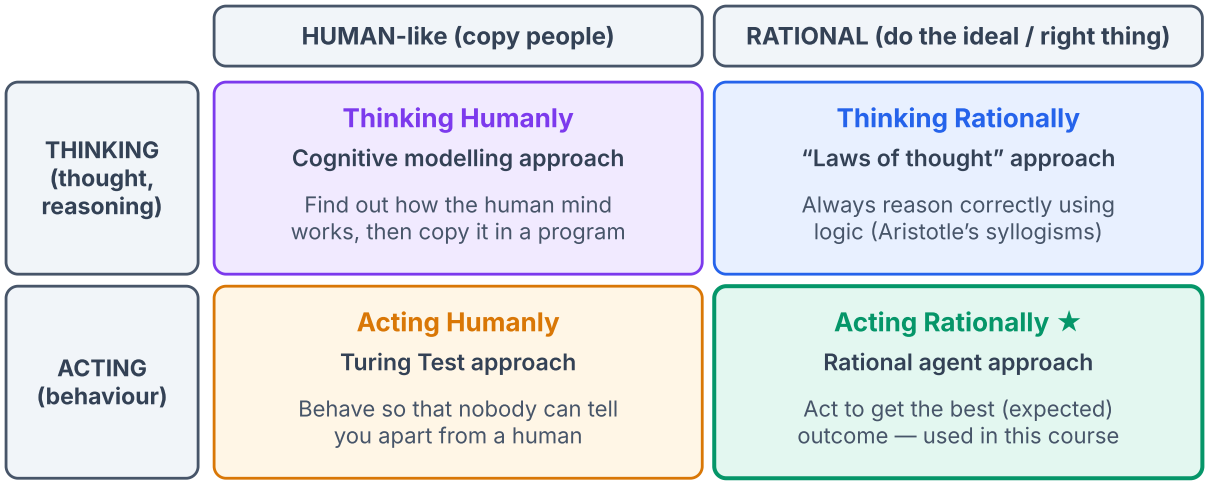


Fig 1.1 The 2 × 2 grid: Thinking/Acting × Human/Rational gives four approaches.

(a) Acting humanly — the Turing Test approach

Plain idea: "If it behaves like a human and you can't tell the difference, call it intelligent." Alan Turing (1950, paper *"Computing Machinery and Intelligence"*) proposed an operational test instead of a philosophical definition.

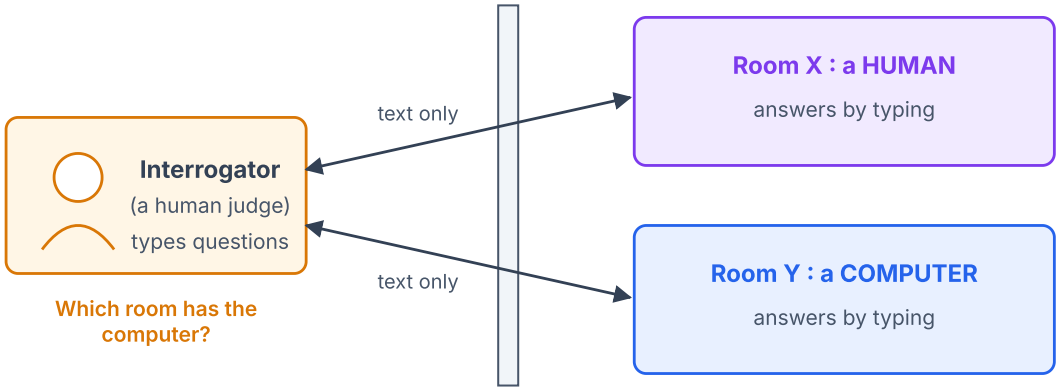


Fig 1.2 The Turing test (imitation game).

1. A human **interrogator** types questions to two hidden parties: a human and a computer.
2. Both reply only through text (a teletype / chat), so voice and looks give nothing away.
3. The computer passes the test if the interrogator cannot reliably tell which reply came from the computer.

To pass the (standard) Turing test, the computer needs **four capabilities**:

Capability	Why it is needed
1. Natural language processing (NLP)	to communicate successfully in English (or any human language)
2. Knowledge representation	to store what it knows or hears
3. Automated reasoning	to use the stored information to answer questions and draw new conclusions
4. Machine learning	to adapt to new circumstances and to detect and extrapolate patterns

The **Total Turing Test** also includes a video signal (so the interrogator can test perceptual abilities) and a hatch to pass physical objects. It needs two more capabilities: **5. Computer vision** (to perceive objects) and **6. Robotics** (to manipulate objects and move about).

MEMORY TRICK — THE 6 CAPABILITIES

"Nice Kids Always Learn … Very Rapidly" → **N**LP, **K**nowledge representation, **A**utomated reasoning, **L**earning (ML) … + **V**ision, **R**obotics for the *Total* Turing test.

Why AI researchers do not focus on passing it: it is more important to study the underlying principles of intelligence than to duplicate a human. ("Aeronautical engineers did not build aeroplanes by copying pigeons so exactly that they fool other pigeons.")

(b) Thinking humanly — the cognitive modelling approach

Plain idea: first find out *how humans actually think*, then write a program that thinks the same way. We can learn how the mind works in three ways: **introspection** (catching our own thoughts as they go by), **psychological experiments** (observing a person in action) and **brain imaging** (observing the brain in action). If the program's input–output behaviour *and timing* match human behaviour, the program's mechanisms may also be operating in humans.

Example: Newell and Simon's **GPS (General Problem Solver)** — they cared more about comparing its reasoning steps with those of human subjects than about simply getting correct answers. This approach led to the field of **cognitive science** (computer models from AI + experimental methods from psychology).

(c) Thinking rationally — the "laws of thought" approach

Plain idea: always reason *correctly*, using the rules of logic. The Greek philosopher **Aristotle** gave **syllogisms** — patterns of argument that always give correct conclusions from correct premises:

> "Socrates is a man; all men are mortal; therefore Socrates is mortal."

The **logicist** tradition in AI hopes to build intelligent systems from such logical programs. **Two main obstacles:**

1. It is not easy to take *informal* knowledge and state it in formal logical notation, especially when the knowledge is less than 100% certain.
2. There is a big difference between solving a problem "in principle" and solving it in practice: even a few hundred facts can exhaust a computer's resources unless it knows which reasoning steps to try first.

(d) Acting rationally — the rational agent approach ★

Plain idea: do the right thing — whatever is expected to give the best result. An **agent** is just something that acts (Latin *agere*, "to do"). A computer agent is expected to do more: operate autonomously, perceive its environment, persist over a long time, adapt to change, and create and pursue goals.

DEFINITION

A **rational agent** is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome.

Why the rational-agent approach is preferred (textbook reasons):

1. **More general than "laws of thought".** Correct inference is only *one* way of acting rationally. Some rational actions involve no inference at all — e.g. pulling your hand back from a hot stove is a *reflex*, faster and better than slow deliberation.
2. **More open to scientific development.** Rationality is *mathematically well defined* and completely general, so we can "unpack" it to design agents that provably achieve it. Human behaviour, in

contrast, is tuned for one specific environment and is the product of a complex, largely unknown evolutionary process.

3. All the skills needed for the Turing test (language, reasoning, knowledge, learning) are still useful — they help an agent act rationally.

Limited rationality: perfect rationality (always doing exactly the right thing) is not feasible in complicated environments because the computation needed is too high; real agents must act appropriately when there is not enough time to do all the computations.

EXAM TIP — "EXPLAIN THE FOUR APPROACHES TO AI" (10M)

- Draw the **2 × 2 grid** (Fig 1.1) first — it earns marks by itself.
- For each approach write: **name of the approach** (cognitive modelling / laws of thought / Turing test / rational agent), **one-line idea**, **one example** (GPS, Aristotle's syllogism, Turing test, reflex vs. deliberation), and a **limitation**.
- End with "The textbook adopts the rational agent approach because it is more general and mathematically well-defined."

COMMON MISTAKES

- Mixing the axes: the Turing test is **acting humanly** (it only checks behaviour), *not* thinking humanly.
- Writing only four capabilities for the **Total** Turing test — it needs **six** (add vision and robotics).
- "Rational = perfect / all-knowing." No: rational means best *expected* outcome given what the agent knows (more in Ch 2).

1.3 Foundations of AI (where the ideas come from)

Discipline	Question it answered for AI	Contribution (examples)
Philosophy	Can formal rules be used to draw valid conclusions? Where does knowledge come from? How does knowledge lead to action?	Aristotle's syllogisms, rationalism, mind as a physical system
Mathematics	What are the formal rules to draw valid conclusions? What can be computed? How do we reason with uncertain information?	Logic (Boole, Frege), algorithms, computability (Turing), intractability / NP-completeness, probability (Bayes)
Economics	How should we make decisions to maximise payoff?	Utility, decision theory, game theory, MDPs
Neuroscience	How do brains process information?	Neurons; the brain causes the mind
Psychology	How do humans and animals think and act?	Behaviourism, cognitive psychology
Computer engineering	How can we build an efficient computer?	The hardware that runs AI programs
Control theory & cybernetics	How can artifacts operate under their own control?	Feedback, minimising an objective function over time
Linguistics	How does language relate to thought?	Computational linguistics / NLP

1.4 A very short history

Year	Milestone
1943	McCulloch & Pitts: first model of artificial neurons.
1950	Turing's paper "Computing Machinery and Intelligence" — the Turing test, machine learning, reinforcement learning ideas.
1956	**Dartmouth workshop** — John McCarthy coins the term **"Artificial Intelligence"**. Birth of AI.
1952–69	Early enthusiasm: Logic Theorist, GPS, LISP (1958), checkers program that learned.
1966–73	A dose of reality: early systems failed to scale (combinatorial explosion).
1969–86	Knowledge-based / **expert systems** (DENDRAL, MYCIN); AI becomes an industry (1980s).
1986–	Return of neural networks (back-propagation); probabilistic reasoning and machine learning grow.
1997	IBM **Deep Blue** beats world chess champion Garry Kasparov.
2011–today	Big data and **deep learning** (image recognition 2012), AlphaGo beats Lee Sedol (2016), large language models.

1.5 What can AI do today? (applications)

Self-driving / robotic vehicles · Speech recognition · Autonomous planning & scheduling · Game playing (chess, Go) · Spam fighting · Logistics planning · Robotics · Machine translation · Medical diagnosis · Recommendation systems · Face / image recognition

EXAM TIP — SHORT ANSWERS

For a 4–5 mark "What is AI?" question: give the **definition**, the **two dimensions**, name the **four approaches** in one line each, and finish with **3–4 applications**. Do not write a long history unless asked.

LIKELY EXAM QUESTIONS — CHAPTER 1

1. Define AI. Explain the four approaches to AI with a neat diagram. **(10M)**
2. What is the Turing test? List the capabilities a computer needs to pass it. What is the Total Turing test? **(6M)**
3. Explain the "laws of thought" approach and its two obstacles. **(4M)**
4. Why is the rational-agent approach preferred over the other approaches? **(5M)**
5. Explain the cognitive modelling approach. **(4M)**
6. List the foundations of AI and the question each discipline answers. **(5M)**
7. List any five applications of AI. **(3M)**

60-SECOND RECAP

- AI = building **intelligent agents** that perceive and act to achieve the best expected outcome.
- Two axes: **thinking vs acting**, **human vs rational** → four approaches.
- Thinking humanly = **cognitive modelling**; thinking rationally = **laws of thought** (logic, syllogisms); acting humanly = **Turing test**; acting rationally = **rational agent** (used in this course).
- Turing test needs **NLP, knowledge representation, automated reasoning, ML**; Total Turing test adds **vision, robotics**.
- Rational-agent approach wins: more general (not only inference) and mathematically well-defined.

2 Intelligent Agents

An agent is anything that senses and acts. This chapter defines a *good* (rational) agent, shows how to describe its task (PEAS), how to classify environments, and the five standard agent designs — all very frequently examined.

2.1 Agents and environments

DEFINITIONS

An **agent** is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

- **Sensor** — the part that takes input from the environment (eyes, camera, keyboard).
- **Actuator** — the part that produces an effect on the environment (hands, motors, screen display).
- **Percept** — the agent's perceptual input *at any given instant*.
- **Percept sequence** — the *complete history* of everything the agent has ever perceived.

Key fact: an agent's choice of action at any instant can depend on its entire percept sequence observed to date, but not on anything it hasn't perceived.

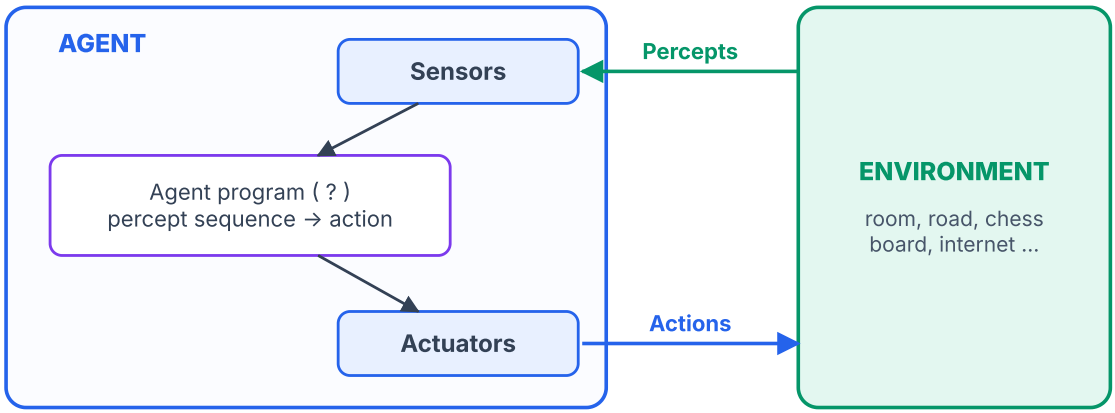


Fig 2.1 Agents interact with environments through sensors and actuators. The "?" is the agent program we must design.

Agent	Sensors	Actuators
Human agent	eyes, ears, nose, skin (touch)	hands, legs, vocal tract (mouth)
Robotic agent	cameras, infrared range finders	various motors (wheels, arms)
Software agent	keystrokes, file contents, received network packets	displaying on screen, writing files, sending network packets

Agent function vs agent program

AGENT FUNCTION (ABSTRACT)

A **mathematical mapping** from *every possible percept sequence* to an action:

$$f : P^* \to A$$

It describes the agent's behaviour completely; in principle it could be written as a (huge) table.

AGENT PROGRAM (CONCRETE)

The **actual code** that implements the agent function and runs on the agent's physical **architecture** (computer + sensors + actuators).

It takes only the **current percept** as input (if it needs history, it must remember it).

MEMORY TRICK

Function = WHAT the agent does (a specification on paper). **Program = HOW** it does it (code in a machine). Agent = Architecture + Program.

2.2 Worked example: the vacuum-cleaner world

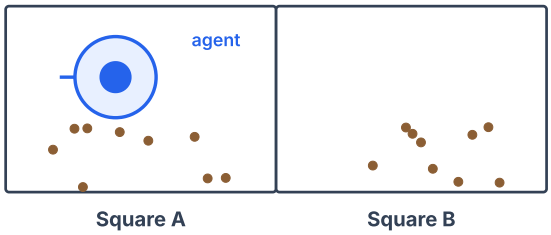


Fig 2.2 A vacuum-cleaner world with just two squares.

- **Squares:** A and B.
- **Percepts:** [location, status], e.g. `[A, Dirty]`.
- **Actions:** *Left*, *Right*, *Suck*, *NoOp* (do nothing).
- **Simple agent function:** "if the current square is dirty, suck; otherwise move to the other square."

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
...	...
[A, Clean], [A, Clean], [A, Dirty]	Suck

Partial tabulation of the agent function.

```
function REFLEX-VACUUM-AGENT([location, status])
          returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

The table on the left is the **agent function**; this 3-line code is an **agent program** implementing it. Notice how much smaller the program is.

2.3 Good behaviour: the concept of rationality ★ Very likely

A **rational agent** is one that *does the right thing*. But what is "right"? We judge an agent by the *consequences* of its behaviour: the actions change the environment through a sequence of states; if that sequence is desirable, the agent performed well. Desirability is captured by a **performance measure**.

DESIGNING A PERFORMANCE MEASURE

Design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.

Example: "amount of dirt cleaned in 8 hours" is a bad measure — a clever agent could clean, dump the dirt back, and clean again! Better: "reward the agent for having a clean floor", e.g. +1 point for each clean square at each time step.

What is rational at any given time depends on FOUR things

1. The **performance measure** that defines the criterion of success.
2. The agent's **prior knowledge** of the environment.
3. The **actions** that the agent can perform.
4. The agent's **percept sequence** to date.

MEMORY TRICK

"PPAP" → **P**erformance measure, **P**rior knowledge, **A**ctions, **P**ercept sequence.

DEFINITION OF A RATIONAL AGENT (LEARN WORD-FOR-WORD)

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

WORKED EXAMPLE — IS THE SIMPLE VACUUM AGENT RATIONAL?

Yes, **under these assumptions**:

- Performance measure: +1 for each clean square at each time step, over 1000 time steps.
- The geography is known in advance, but the dirt distribution and the agent's initial location are not. Clean squares stay clean; *Suck* cleans the current square; *Left*/*Right* move the agent (no effect at the edge).
- The only actions are *Left*, *Right*, *Suck*; the agent correctly perceives its location and whether it contains dirt.

Under different circumstances the **same agent becomes irrational**: e.g. if each movement costs −1 point, then once both squares are clean it keeps oscillating needlessly; a better agent would do *NoOp*. If clean squares can become dirty again, it should check them occasionally.

Rationality is NOT omniscience

An **omniscient** agent knows the *actual* outcome of its actions and can act accordingly — impossible in reality. *Example:* you look both ways and cross the road; a cargo door falls from a passing aeroplane and flattens you. You were still **rational** (given what you could perceive), just not omniscient.

Rational	Omniscient	Perfect
maximizes **expected** performance, given percepts so far	knows the **actual** outcome of every action in advance	maximizes **actual** performance (needs omniscience)

Because rationality depends only on the percept sequence *to date*, a rational agent must also do:

- **Information gathering** — doing actions to modify future percepts (e.g. *looking* both ways before crossing). **Exploration** of an unknown environment is one kind of information gathering.

- **Learning** — learn as much as possible from what it perceives. Prior knowledge can be modified and augmented with experience.
- **Autonomy** — an agent is autonomous to the extent that it relies on its own percepts (and learning) rather than on the prior knowledge of its designer. Counter-examples from nature: the *dung beetle* keeps "plugging" its nest even after the dung ball is removed; the *sphex wasp* repeats its fixed routine endlessly when the caterpillar is moved. Their fixed behaviour fails when assumptions break. A truly rational agent should become independent of its prior knowledge after sufficient experience.

COMMON MISTAKES

- "A rational agent never makes mistakes." — Wrong. Rational ≠ omniscient ≠ perfect. It maximises **expected**, not actual, performance.
- Forgetting "*given the percept sequence and built-in knowledge*" in the definition — examiners look for these words.
- Confusing agent *function* (a mapping) with agent *program* (code).

2.4 The nature of environments

A **task environment** is the "problem" to which a rational agent is the "solution". Before designing an agent we must specify it as fully as possible.

PEAS description ★ Very likely

DEFINITION

PEAS = **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors — the four things that specify a task environment. Writing PEAS is always the first step in designing an agent.

WORKED EXAMPLE — PEAS FOR AN AUTOMATED TAXI DRIVER (THE TEXTBOOK'S MAIN EXAMPLE)

PEAS	Automated taxi driver	Why (plain words)
Performance measure	Safe, fast, legal, comfortable trip; maximize profits; minimize fuel consumption and wear; minimize impact on other road users	Goals often **conflict** (fast vs safe) → trade-offs needed
Environment	Roads (lanes, highways), other traffic, pedestrians, stray animals, road works, police cars, puddles, potholes, customers, weather	Everything the taxi deals with
Actuators	Steering, accelerator, brake, signal (indicator), horn, display / speech to talk to passengers	How it changes the world
Sensors	Cameras, sonar / radar / LIDAR, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard / microphone	How it observes the world

Agent type	Performance measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Colour pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry
Vacuum-cleaner robot	Cleanliness, efficiency (time, energy), battery life, safety	Rooms, floor (wood/carpet), furniture, obstacles, dirt	Wheels, brushes, vacuum extractor	Camera, dirt-detection sensor, bump / cliff sensors, infrared wall sensor

EXAM TIP — PEAS QUESTIONS (5M)

- Draw a 4-row table with headings **P, E, A, S**; give **3–5 items** in each row.
- Be ready for *new* agents too (e.g. a robot soccer player, an online shopping assistant, a drone, a mathematician's theorem-proving assistant). Ask yourself: *How is success scored? Where does it live? What can it move/output? What can it read/sense?*
- Don't mix A and S: actuators **do**, sensors **sense**. "Camera" is never an actuator; "display" is never a sensor.

Properties (dimensions) of task environments ★ Very likely

Use the "test question" in the second column to decide each property quickly.

Property	Test question (plain English)	Meaning & examples
Fully observable vs **partially observable**	Can the sensors see *everything relevant* to choosing an action, at every moment?	**Fully:** sensors give the complete state (chess — the whole board is visible). No need to keep internal state. **Partially:** noisy/inaccurate sensors or parts of the state missing (vacuum agent with only a local dirt sensor can't tell whether the other square is dirty; a taxi can't see what other drivers are thinking; poker — opponents' cards hidden). **Unobservable:** no sensors at all.
Single agent vs **multiagent**	Is there another agent whose performance depends on my behaviour?	**Single:** crossword puzzle. **Multiagent:** chess (**competitive** — opponent maximises its score by minimising mine); taxi driving (**partially cooperative** — avoiding collisions helps everyone; **partially competitive** — only one car can take a parking space). Communication and randomized behaviour often matter in multiagent settings.
Deterministic vs **stochastic** (nondeterministic)	Is the next state *completely determined* by the current state and my action?	**Deterministic:** crossword, chess (ignoring the opponent). **Stochastic:** outcome uncertain, quantified by probabilities (taxi — tyres blow out, traffic unpredictable; backgammon — dice). **Nondeterministic:** actions have possible outcomes but *no probabilities* attached. An environment is **uncertain** if it is not fully observable or not deterministic.

Episodic vs **sequential**	Does my current decision affect future decisions?	**Episodic:** experience is divided into atomic episodes (perceive, then one action); the next episode does not depend on earlier actions (part-picking robot judging each part on an assembly line; image classification). **Sequential:** current decision can affect all future decisions (chess, taxi driving). Episodic is much simpler — no need to think ahead.
Static vs **dynamic** (vs **semidynamic**)	Can the world change *while I am thinking*?	**Static:** world waits for the agent (crossword). **Dynamic:** world keeps changing (taxi driving — other cars move while the agent decides). **Semidynamic:** the environment itself doesn't change with time, but the agent's *performance score* does (chess played with a clock).
Discrete vs **continuous**	Are states, time, percepts and actions *countable steps* or smooth values?	**Discrete:** chess — finite number of distinct states, discrete set of percepts and actions. **Continuous:** taxi driving — speed and location sweep through a range of continuous values smoothly over time; steering angles are continuous.
Known vs **unknown**	Does the agent (designer) know the "laws of physics" of the environment?	**Known:** the outcomes (or outcome probabilities) of all actions are given. **Unknown:** the agent must learn how the world works. Note: known ≠ fully observable — solitaire card games are known but partially observable; a new video game can be fully observable (the whole screen is visible) but unknown (you don't know what the buttons do).

KEY POINT

The **hardest** case is: partially observable, multiagent, stochastic, sequential, dynamic, continuous and unknown — e.g. taxi driving (except that the environment is mostly known).

MEMORY TRICK — THE 7 DIMENSIONS

"Oh, All Dogs Eat Some Dog Kibble" → **O**bservable? · **A**gents (single/multi) · **D**eterministic? · **E**pisodic? · **S**tatic? · **D**iscrete? · **K**nown?

Classification of standard task environments

Task environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

COMMON MISTAKES

- **Chess with a clock** is **semi**dynamic (not dynamic): the board doesn't change while you think, but your remaining time does.
- **Image analysis** is **episodic** (each image is independent) and semidynamic; **backgammon** is fully observable but **stochastic** (dice).
- "Partially observable" is caused by *sensors*; "stochastic" is caused by *actions/world dynamics*. Keep them separate.

EXAM TIP — "CLASSIFY THE ENVIRONMENT OF X"

Write **all 7 properties** as a list with **a one-line reason for each** (e.g. "Partially observable — the taxi cannot see around corners"). Reasons carry the marks; a bare table of words gets half credit.

2.5 The structure of agents ★ Very likely

$$\text{Agent} = \text{Architecture} + \text{Program}$$

The **architecture** is the computing device with physical sensors and actuators; the **program** implements the agent function. All agent programs here have the same skeleton: *take the current percept from the sensors, return an action to the actuators*.

Table-driven agent (and why it fails)

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by percept
                     sequences, initially fully specified
  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

It literally stores the agent function as a table. If P is the set of possible percepts and T the lifetime, the table has $\sum_{t=1}^{T} |P|^t$ entries — for chess **at least 10^{150}** entries (the observable universe has fewer than 10^{80} atoms). It is doomed because:

1. no physical agent has the space to store the table;
2. the designer would not have time to create it;
3. no agent could ever learn all the right entries from experience;
4. even for a simple environment, there is no guidance about how to fill in the entries.

The key challenge of AI: write programs that produce rational behaviour from a small program rather than a vast table. Four basic kinds of agent programs do this, plus the learning agent:

MEMORY TRICK — AGENT TYPES IN INCREASING POWER

"Some Monkeys Grab Useful Leaves" → **S**imple reflex → **M**odel-based reflex → **G**oal-based → **U**tility-based → **L**earning agent.

(1) Simple reflex agent

Plain idea: reacts to *what it sees right now* using **condition–action rules** (if–then rules), ignoring all history. Like blinking when something flies at your eye.

if *car-in-front-is-braking* then *initiate-braking*

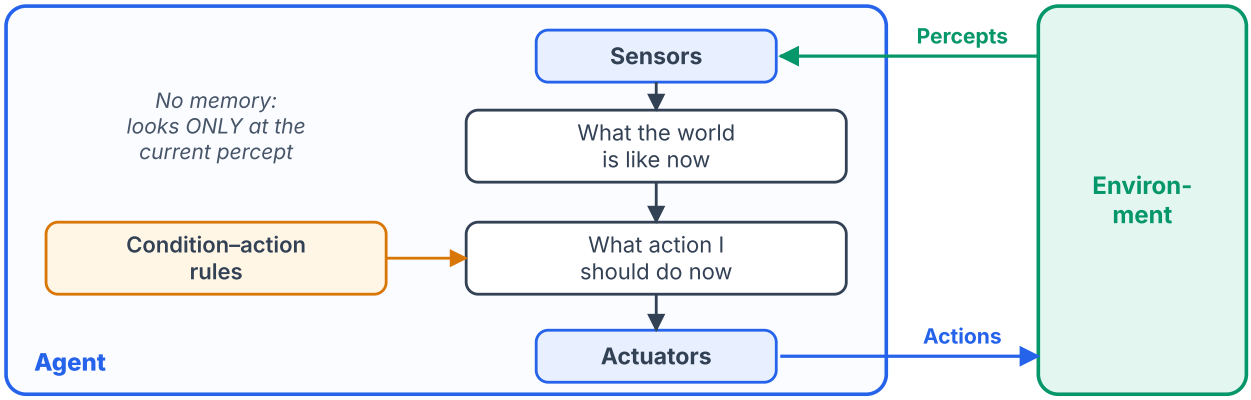


Fig 2.3 Schematic diagram of a simple reflex agent (white boxes = current internal state of the decision process; amber box = background knowledge used).

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules
  state  ← INTERPRET-INPUT(percept)
  rule   ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

- **Example:** REFLEX-VACUUM-AGENT (Section 2.2).
- **Advantage:** very simple, fast, small.
- **Limitation:** works only if the correct decision can be made on the basis of the current percept alone, i.e. only if the environment is fully observable. Even a little unobservability causes trouble.
- **Infinite loops:** a vacuum agent with no location sensor, in a clean square, may keep moving *Left* forever. Escape: **randomize** the action (e.g. flip a coin to choose Left/Right).

(2) Model-based reflex agent

Plain idea: keeps an **internal state** (memory) of the parts of the world it can't see now. Like a driver who remembers that a car is in the blind spot even when it is not visible in the mirror.

To update its internal state it needs two kinds of knowledge (together called a **model** of the world):

- **Transition model** — how the world changes over time: (a) how the world evolves independently of the agent (an overtaking car gets closer), and (b) how the agent's own actions affect the world (turning the wheel clockwise turns the car right).
- **Sensor model** — how the state of the world is reflected in the agent's percepts (e.g. when it is dark, car headlights appear in the camera image).

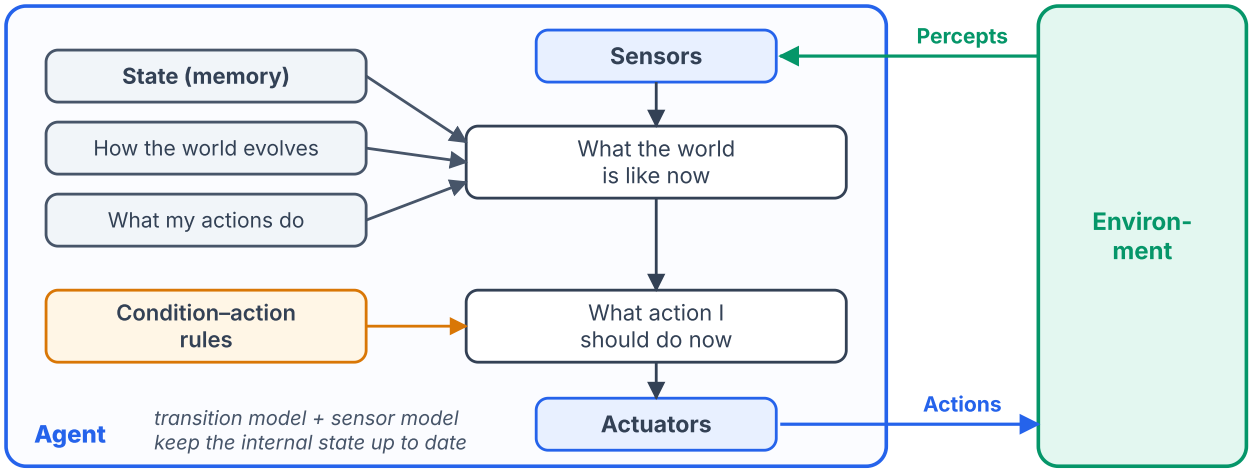


Fig 2.4 A model-based reflex agent: it combines the current percept with its old internal state to estimate the current world.

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              transition_model, how the next state depends on the
                                current state and action
              sensor_model, how the current world state is
                            reflected in the agent's percepts
              rules, a set of condition–action rules
              action, the most recent action, initially none
  state  ← UPDATE-STATE(state, action, percept,
                        transition_model, sensor_model)
  rule   ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

The internal state is usually a *best guess* — in a partially observable world the agent can rarely know the exact state.

(3) Goal-based agent

Plain idea: knowing the current state is not always enough — at a road junction the taxi can turn left, right or go straight. The correct decision depends on **where it is trying to get to**, i.e. its **goal**. The agent combines its model with goal information and asks “what will happen if I do action A?” and “will that bring me closer to my goal?”

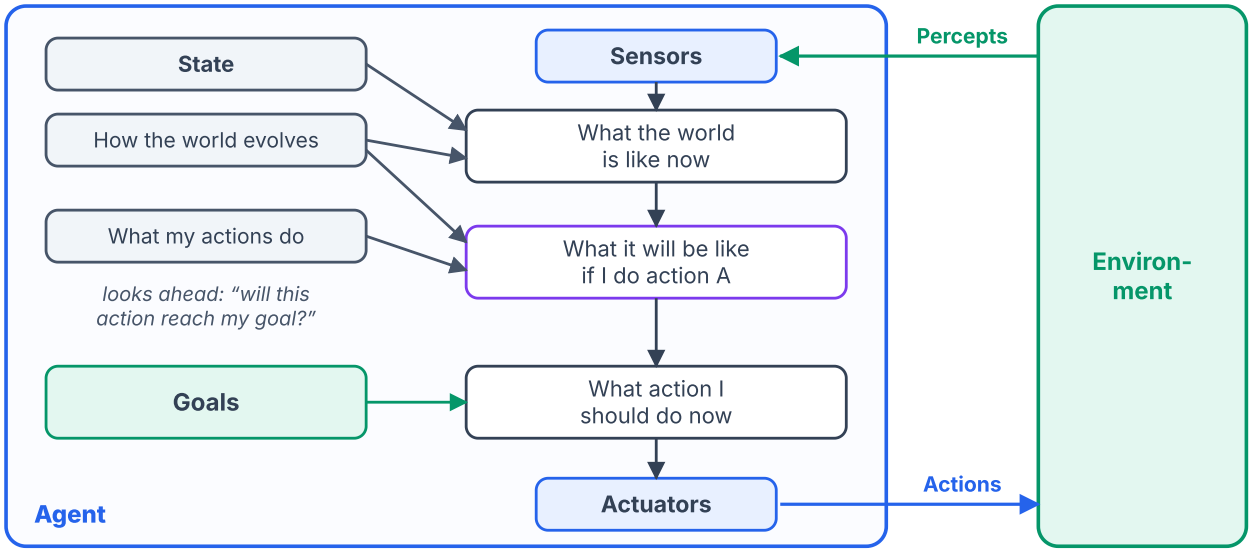


Fig 2.5 A model-based, goal-based agent: it keeps track of the world state and a set of goals, and chooses an action that will (eventually) achieve them.

- Involves **looking into the future** — **search** (Ch 3) and **planning** find action sequences that reach the goal.
- **More flexible** than a reflex agent: the knowledge behind decisions is explicit and can be changed. E.g. if it starts to rain, the agent updates its knowledge of how effectively its brakes work and automatically changes its behaviour; to go to a new destination we just change the goal — no need to rewrite all the rules.
- **Less efficient** than a reflex agent (it has to think ahead).

(4) Utility-based agent

Plain idea: goals only give a crude binary distinction "happy / unhappy". Many routes reach the destination, but some are quicker, safer, more reliable, or cheaper. A **utility function** gives a *number* saying **how happy (desirable)** a state is.

> **DEFINITION**
>
> A **utility function** is an internalization of the performance measure: $U(\text{state}) \rightarrow \text{real number}$. A rational utility-based agent chooses the action that maximizes the expected utility of the action outcomes (the utility it expects to get, on average, given the probabilities of each outcome).

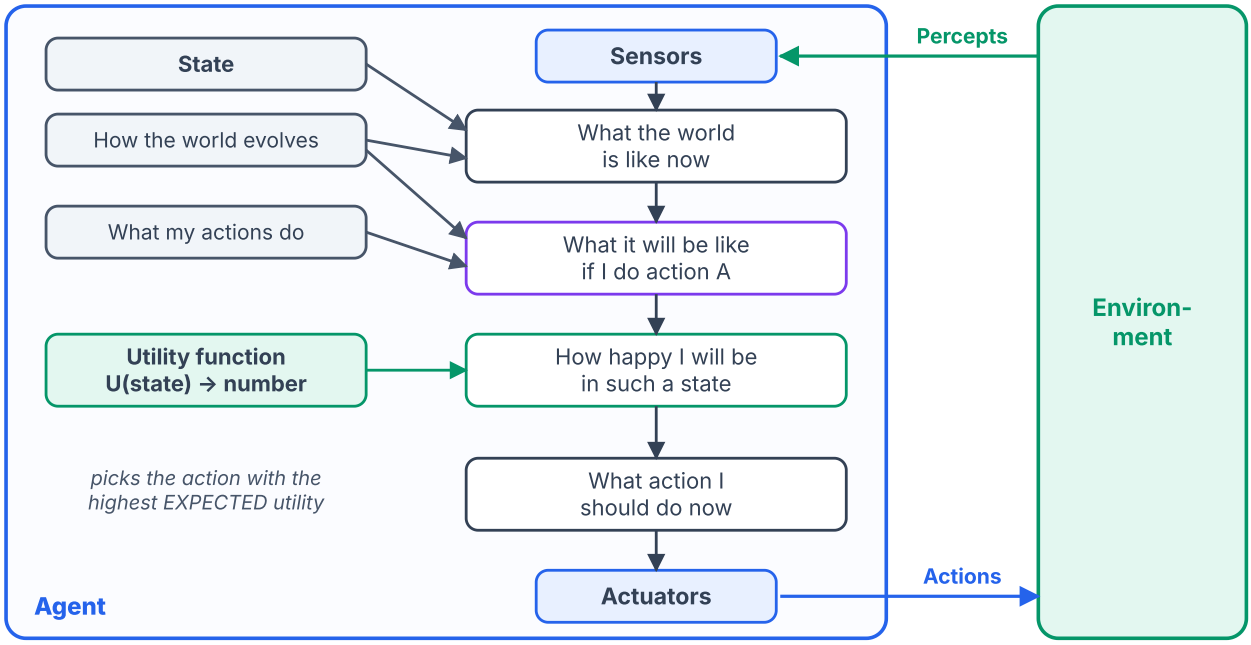


Fig 2.6 A model-based, utility-based agent.

Utility helps where goals are inadequate: (1) when there are **conflicting goals** (speed vs safety) — utility specifies the right trade-off; (2) when there are **several goals, none achievable with certainty** — utility weighs the likelihood of success against the importance of each goal.

(5) Learning agent

Plain idea: any of the above agents can be made to **improve itself with experience**. Learning allows the agent to operate in initially unknown environments and become more competent than its initial knowledge alone would allow. (Turing's proposal: build learning machines and then teach them.)

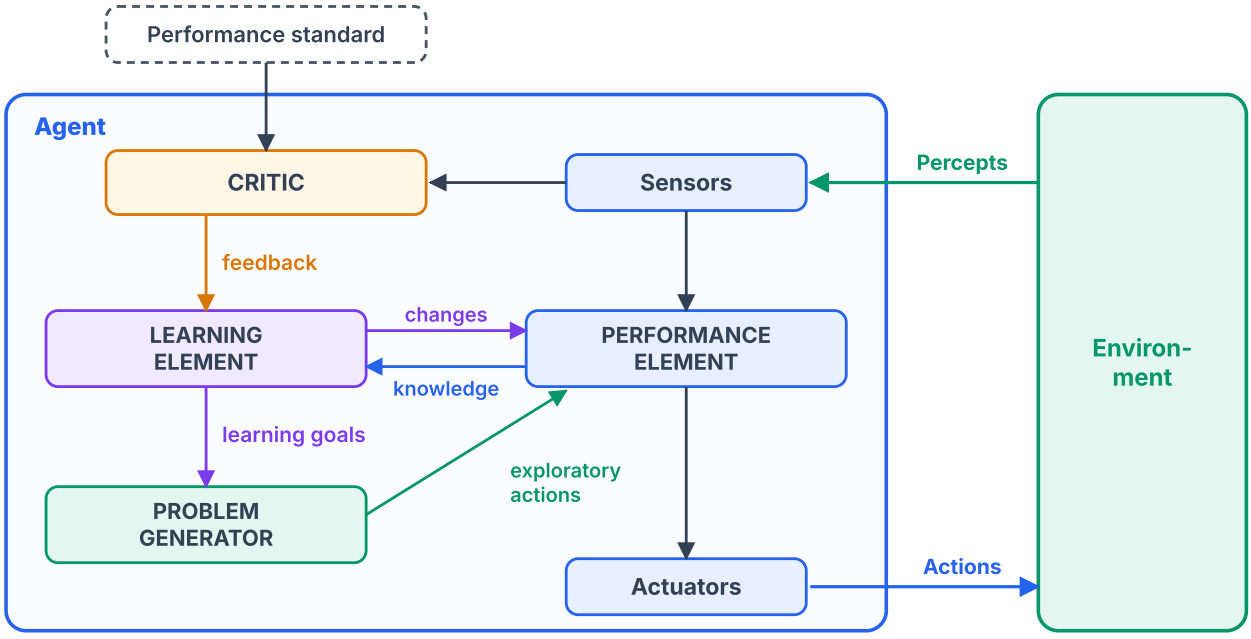


Fig 2.7 A general learning agent with its four conceptual components.

Component	Role	Taxi example
Performance element	Selects external actions (this is the "whole agent" we discussed earlier: takes percepts, decides actions)	Knowledge & procedures for driving
Critic	Tells the learning element how well the agent is doing **with respect to a fixed performance standard** (percepts alone don't say whether the agent is doing well)	Observes other drivers' shocked reactions after a quick left turn across lanes
Learning element	Makes improvements: uses feedback from the critic to decide how the performance element should be modified	Forms a rule "quick left turns across traffic are bad"
Problem generator	Suggests **exploratory actions** that lead to new and informative experiences (may be sub-optimal in the short run, but better in the long run)	"Try the brakes on a different road surface"

Comparison of agent types

Agent type	Uses memory (internal state)?	Decides using	Strength	Weakness / limitation
Table-driven	Whole percept sequence	Table lookup	Trivially correct in theory	Table astronomically large
Simple reflex	No	Condition–action rules on current percept	Fast, simple	Needs fully observable env.; infinite loops
Model-based reflex	Yes (state + model)	Rules on estimated current state	Handles partial observability	No explicit goals; still rule-bound
Goal-based	Yes	Goals + prediction (search/planning)	Flexible; goals easy to change	Slower; goals are only yes/no
Utility-based	Yes	Maximize expected utility	Trade-offs, uncertainty, conflicting goals	Utility hard to design; heavy computation
Learning	Yes	Any of the above + improvement from feedback	Works in unknown environments; improves	Needs experience; exploration costs

How states are represented inside an agent

Representation	What a state looks like	Used by
Atomic	an indivisible "black box" with no internal structure (e.g. just the city name "Arad")	search & game playing (Ch 3), HMMs, MDPs
Factored	a fixed set of variables/attributes, each with a value (fuel = 40%, GPS = ..., light = on)	CSPs, **propositional logic** (Ch 4), planning, Bayesian networks
Structured	objects and the relationships between them (truck *is backing into* driveway, cow *blocking* it)	first-order logic, relational databases, NLP

EXAM TIP — "EXPLAIN THE TYPES OF AGENTS WITH NEAT DIAGRAMS" (10M)

- This is one of the most common 10-mark questions. For **each** type give: one-line idea, **diagram**, pseudocode (at least for simple reflex and model-based), an example, and a limitation.
- Practise drawing the diagrams quickly: **Agent box on the left, Environment box on the right, Percepts arrow in at the top (Sensors), Actions arrow out at the bottom (Actuators)**. Then fill the inside: simple reflex (2 boxes + rules), model-based (+ State, How the world evolves, What my actions do), goal-based (+ What it will be like if I do A, Goals), utility-based (+ How happy I will be, Utility).
- For the learning agent, label all four components and the arrows *feedback, changes, knowledge, learning goals*.

COMMON MISTAKES

- Saying a simple reflex agent has "memory" — it does not; that is the model-based agent.
- Saying the *critic* changes the agent — the critic only gives feedback; the *learning element* makes changes.
- Calling the utility function a goal. Goal = yes/no; utility = degree of happiness (a number).

LIKELY EXAM QUESTIONS — CHAPTER 2

1. Define agent, percept, percept sequence, agent function and agent program. Explain with the vacuum-cleaner world. **(6M)**
2. What is a rational agent? What does rationality depend on? Differentiate rationality and omniscience. **(6M)**
3. Explain autonomy, information gathering and learning with respect to rational agents. **(5M)**
4. Give the PEAS description for an automated taxi driver / medical diagnosis system / part-picking robot / English tutor. **(5M)**
5. Explain the properties of task environments with examples. **(10M)**
6. Classify crossword, chess with a clock, poker, taxi driving and medical diagnosis along all dimensions. **(5M)**
7. Why does the table-driven agent fail? **(4M)**
8. Explain simple reflex and model-based reflex agents with diagrams and pseudocode. **(10M)**
9. Explain goal-based and utility-based agents with diagrams. How do they differ? **(8M)**
10. Explain the learning agent with a neat diagram. **(6M)**

60-SECOND RECAP

- Agent = perceives via **sensors**, acts via **actuators**; agent function (percept sequence → action) vs agent program (code). Agent = architecture + program.
- Rationality depends on **performance measure, prior knowledge, actions, percept sequence**; rational maximizes **expected** performance; rational ≠ omniscient. Needs information gathering, learning, autonomy.
- **PEAS** = Performance, Environment, Actuators, Sensors (taxi example).
- 7 environment dimensions: observable, agents, deterministic, episodic, static, discrete, known.
- Agent types: **simple reflex** (rules on current percept), **model-based** (internal state, transition + sensor model), **goal-based** (look ahead to goal), **utility-based** (maximize expected utility), **learning** (performance element, critic, learning element, problem generator).

3 Problem Solving by Searching

How can an agent find a sequence of actions that reaches its goal? By formulating the problem precisely and then **searching** through possible action sequences. This chapter carries the most marks in Unit I: problem formulation, the six uninformed strategies, A* and heuristics.

3.1 Problem-solving agents

A **problem-solving agent** is a *goal-based agent* that uses **atomic** state representations (a state is just a name such as "Arad"). **Story used throughout:** an agent is on holiday in **Arad** (Romania) and has a non-refundable flight leaving from **Bucharest** tomorrow. It must find a route.

1. **Goal formulation** — decide what you want, based on the current situation and the performance measure: *be in Bucharest*. Goals limit the objectives the agent tries to achieve.
2. **Problem formulation** — decide what *actions* and *states* to consider, given the goal (e.g. "drive from one city to a neighbouring city", not "move the left foot 1 cm").
3. **Search** — simulate sequences of actions in the model until one reaching the goal is found; the result is a **solution** (an action sequence).
4. **Execution** — carry out the actions in the solution one by one.

"**Formulate, search, execute**." Here we assume the environment is fully observable, discrete, known and deterministic, so the solution is a fixed sequence of actions and the agent can execute it with its "eyes closed" (an **open-loop** system: percepts are ignored while executing).

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, some description of the current world state
              goal, a goal, initially null
              problem, a problem formulation
  state ← UPDATE-STATE(state, percept)
  if seq is empty then
      goal    ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq     ← SEARCH(problem)
      if seq = failure then return a null action
  action ← FIRST(seq)
  seq    ← REST(seq)
  return action
```

Well-defined problems and solutions ★ Very likely

THE 5 COMPONENTS OF A PROBLEM (LEARN THIS LIST)

1. **Initial state** — the state the agent starts in. *In(Arad)*.
2. **Actions** — ACTIONS(*s*) returns the set of actions applicable in state *s*. From *In(Arad)*: {*Go(Sibiu)*, *Go(Timisoara)*, *Go(Zerind)*}.
3. **Transition model** — RESULT(*s*, *a*) returns the state that results from doing action *a* in state *s* (its **successor**). RESULT(*In(Arad)*, *Go(Zerind)*) = *In(Zerind)*.
4. **Goal test** — decides whether a given state is a goal. Either an explicit set ({*In(Bucharest)*}) or an abstract property ("checkmate" in chess).
5. **Path cost** — a function that assigns a numeric cost to each path; it is the sum of **step costs** *c*(*s*, *a*, *s'*) (assumed ≥ 0). In Romania: distance in km.

State space	the set of all states reachable from the initial state by any sequence of actions; it forms a **directed graph** (nodes = states, links = actions). Initial state + actions + transition model together define it implicitly.
Path	a sequence of states connected by a sequence of actions.
Solution	an action sequence that leads from the initial state to a goal state.
Optimal solution	the solution with the lowest path cost among all solutions.
Abstraction	removing irrelevant detail from the representation (weather, radio, scenery). Valid if every abstract solution can be expanded into a real one; useful if the abstract actions are easier than the original problem.

MEMORY TRICK — 5 COMPONENTS

"I Act To Get Paid" → **I**nitial state, **A**ctions, **T**ransition model, **G**oal test, **P**ath cost.

3.2 Example problems

Toy problems illustrate or exercise methods; they have concise, exact descriptions so researchers can compare algorithms. **Real-world problems** are ones people actually care about; they have no single agreed description.

(1) Vacuum world ★ Likely

Component	Formulation
States	agent location × dirt status of each square: $2 \times 2^2 = 8$ states (with n squares: $n \cdot 2^n$)
Initial state	any state can be the initial state
Actions	*Left*, *Right*, *Suck*
Transition model	actions have their expected effects, except: *Left* in the leftmost square, *Right* in the rightmost square and *Suck* in a clean square have no effect
Goal test	all squares are clean
Path cost	each step costs 1

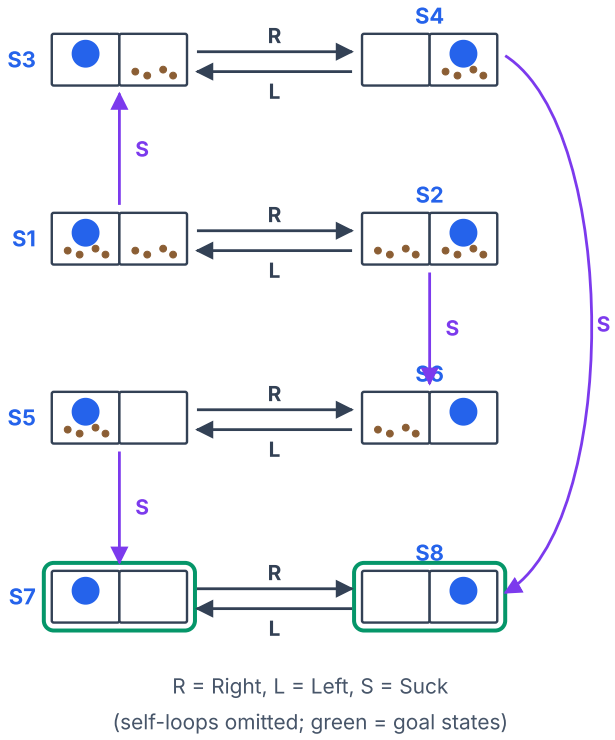


Fig 3.1 State space of the vacuum world (S7, S8 are goals).

(2) The 8-puzzle ★ Likely

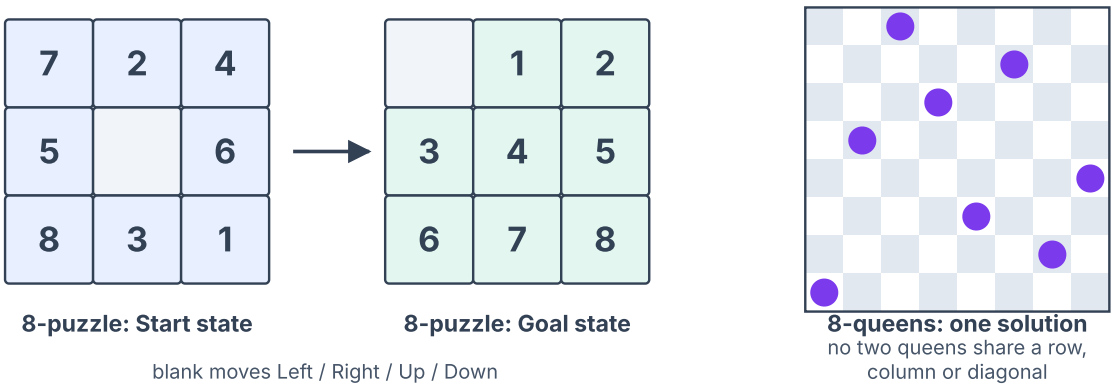


Fig 3.2 A typical 8-puzzle instance (start and goal) and one solution of the 8-queens problem.

A 3×3 board with 8 numbered tiles and one blank. A tile next to the blank can slide into it. Goal: reach the goal configuration.

Component	Formulation
States	location of each of the 8 tiles and the blank in the 9 squares
Initial state	any state (any given goal can be reached from exactly **half** of the possible initial states)
Actions	movements of the **blank**: *Left, Right, Up, Down* (fewer when the blank is at an edge or corner)
Transition model	returns the new board; e.g. applying *Left* to the start state above swaps the 5 and the blank
Goal test	does the state match the goal configuration?
Path cost	each step costs 1 (path cost = number of moves)

Size: the 8-puzzle has 9!/2 = 181,440 reachable states; the 15-puzzle (4×4) about 1.3 trillion; the 24-puzzle (5×5) about 10^{25}. The sliding-block family is **NP-complete**, a standard test bed for heuristic search.

(3) The 8-queens problem

Place 8 queens on a chessboard so that **no queen attacks another** (no two in the same row, column or diagonal). Two kinds of formulation:

- **Incremental formulation** — operators *add* a queen to the state, starting from an empty board.
- **Complete-state formulation** — start with all 8 queens on the board and *move* them around (used by local search).

	Naive incremental formulation	Better incremental formulation
States	any arrangement of 0–8 queens on the board	arrangements of n queens ($0 \le n \le 8$), one per column in the leftmost n columns, none attacking another
Initial state	no queens on the board	no queens on the board
Actions	add a queen to any empty square	add a queen to any square in the leftmost empty column such that it is not attacked
Transition / goal	returns the board with the queen added / 8 queens on the board, none attacked. (Path cost is irrelevant: only the final state matters.)	
Size	64 × 63 × ... × 57 ≈ **1.8×10^{14}** sequences	only **2,057** states — good formulation matters!

(4) Route finding in Romania ★ Very likely

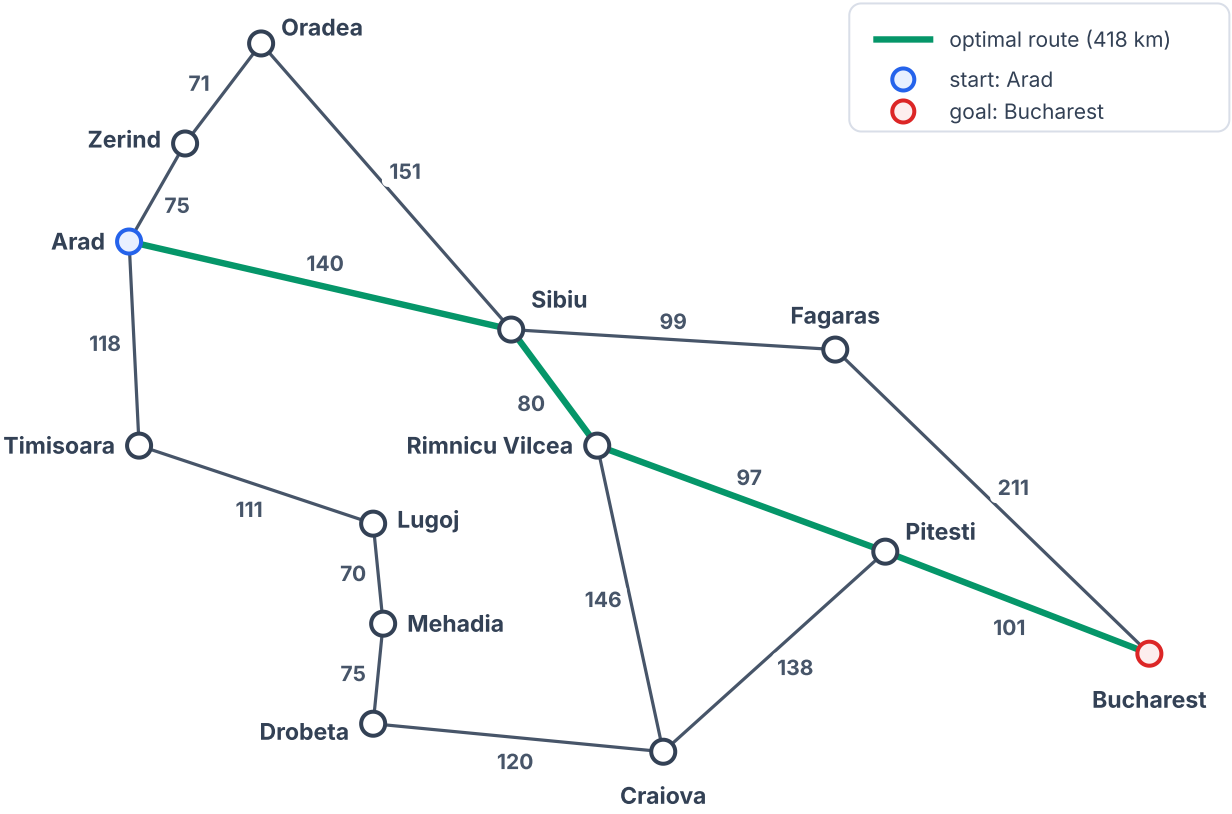


Fig 3.3 Part of the road map of Romania (distances in km). Used for greedy best-first search and A.*

Component	Romania formulation
Initial state	*In(Arad)*
Actions	*Go(city)* for each neighbouring city
Transition	RESULT(*In(x)*, *Go(y)*) = *In(y)*
Goal test	state = *In(Bucharest)*
Path cost	sum of road distances (km)

Straight-line distance to Bucharest, $h_{SLD}(n)$			
Arad	**366**	Mehadia	**241**
Bucharest	**0**	Oradea	**380**
Craiova	**160**	Pitesti	**100**
Drobeta	**242**	Rimnicu Vilcea	**193**
Fagaras	**176**	Sibiu	**253**
Lugoj	**244**	Timisoara	**329**
		Zerind	**374**

Real-world problems (list with one line each)

Route finding	car navigation, routing video streams in networks, military operations planning, **airline travel planning** (state = location + current time + fare bases + domestic/international; cost = money, waiting time, flight time, seat quality ...)
Touring problems	visit every city at least once and return; the **travelling salesperson problem (TSP)**: visit each city exactly once with the shortest tour (NP-hard)
VLSI layout	position millions of components and connections on a chip (cell layout, channel routing) to minimise area, delays, stray capacitances
Robot navigation	route finding in continuous space with many degrees of freedom
Automatic assembly sequencing	find an order in which to assemble the parts of an object; related: protein design (sequence of amino acids that folds correctly)
Internet searching	software agents looking for answers to questions or related information

EXAM TIP — "FORMULATE THE PROBLEM" QUESTIONS (5M)

Always answer with the **5 components** as a table: **States, Initial state, Actions, Transition model, Goal test, Path cost** (states first, as the textbook does). Add the **state-space size** if you can (8 for vacuum, 181,440 for 8-puzzle, 2,057 for good 8-queens). New problems (water jug, missionaries & cannibals, tic-tac-toe) are answered the same way.

3.3 Searching for solutions

A solution is an action sequence, so search algorithms consider many possible action sequences. These form a **search tree**:

- **Root** = initial state; **branches** = actions; **nodes** = states in the state space.
- **Expanding** a node = applying every legal action to its state, thereby **generating** a new set of states (its **child** nodes; it is their **parent**).
- **Leaf node** = a node with no children (yet). The **frontier** (open list) = the set of all leaf nodes available for expansion.
- **Search strategy** = the rule that decides *which frontier node to expand next*. This is the only thing that differs between BFS, DFS, UCS, A* ...

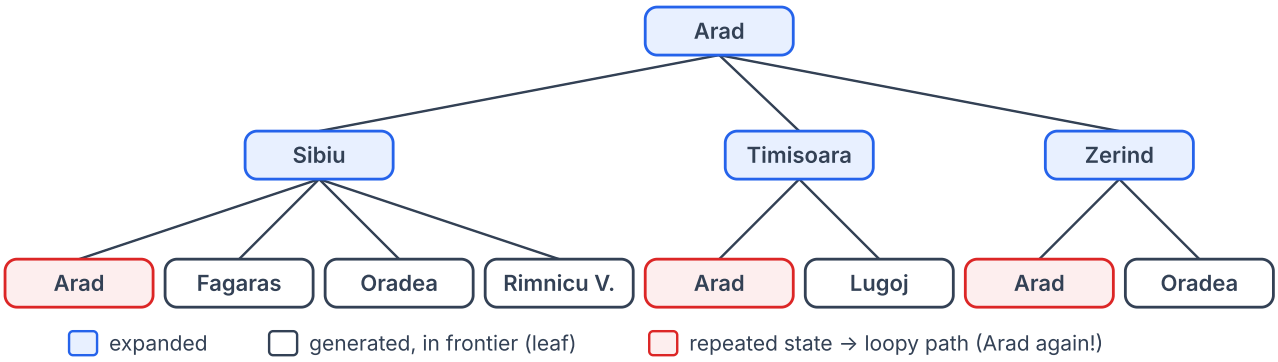


Fig 3.4 Partial search tree for Arad → Bucharest after expanding Arad, then Sibiu, Timisoara and Zerind. Notice Arad appearing again: a loopy path.

Repeated states. The tree contains **Arad → Sibiu → Arad**, a **loopy path**; because of loops the search tree can be *infinite* even though the state space has only 20 cities. Loopy paths are a special case of **redundant paths** (more than one way to reach the same state, e.g. Arad–Sibiu (140 km) vs Arad–Zerind–Oradea–Sibiu (297 km)). "Algorithms that forget their history are doomed to repeat it." The fix is an **explored set** (closed list) that remembers every expanded node.

```
function TREE-SEARCH(problem) returns a
solution, or failure
  initialize the frontier using the initial
     state of problem
  loop do
    if the frontier is empty then return
failure
    choose a leaf node and remove it from
       the frontier
    if the node contains a goal state then
       return the corresponding solution
    expand the chosen node, adding the
       resulting nodes to the frontier
```

```
function GRAPH-SEARCH(problem) returns a
solution, or failure
  initialize the frontier using the initial
     state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return
failure
    choose a leaf node and remove it from the
frontier
    if the node contains a goal state then
       return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the
resulting
       nodes to the frontier only if not in
the
       frontier or explored set
```

Tree search may revisit states (simple, little memory, but may loop); **graph search** never expands a state twice. The frontier *separates* the explored region of the state space from the unexplored region.

Infrastructure: nodes and queues

Field of node *n*	Meaning
`n.STATE`	the state this node corresponds to
`n.PARENT`	the node that generated it
`n.ACTION`	the action applied to the parent
`n.PATH-COST`	$g(n)$, cost of the path from the root to n

Following PARENT pointers back from a goal node gives the solution path. A **node** is a bookkeeping data structure; a **state** is a world configuration — two different nodes can contain the same state.

```
function CHILD-NODE(problem, parent, action)
         returns a node
  return a node with
    STATE  = problem.RESULT(parent.STATE,
action),
    PARENT = parent, ACTION = action,
    PATH-COST = parent.PATH-COST +
        problem.STEP-COST(parent.STATE,
action)
```

Frontier = a **queue** with EMPTY?, POP, INSERT. Types: **FIFO** queue (pops oldest), **LIFO** queue / stack (pops newest), **priority queue** (pops the element with the best priority).

Measuring problem-solving performance ★ Very likely

- **Completeness** — is the algorithm guaranteed to find a solution when there is one?
- **Optimality** — does it find the optimal (lowest path cost) solution?
- **Time complexity** — how long does it take? (number of nodes generated)
- **Space complexity** — how much memory? (max nodes stored)
- ***b*** = **branching factor**: maximum number of successors of any node.
- ***d*** = **depth** of the *shallowest* goal node (number of steps from the root).
- ***m*** = **maximum length** of any path in the state space.
- **Search cost** (time, memory) + **path cost** of the solution = **total cost**.

MEMORY TRICK

"COST" → **C**ompleteness, **O**ptimality, **S**pace, **T**ime. And **b** = **b**readth of the tree, **d** = **d**epth of the goal, **m** = **m**aximum depth.

3.4 Uninformed (blind) search strategies ★ Very likely

Uninformed strategies have no information about states beyond the problem definition: all they can do is generate successors and tell a goal state from a non-goal state. They differ only in the **order** in which nodes are expanded. (This section is a self-study topic in your syllabus, so it is explained slowly with one example for all six strategies.)

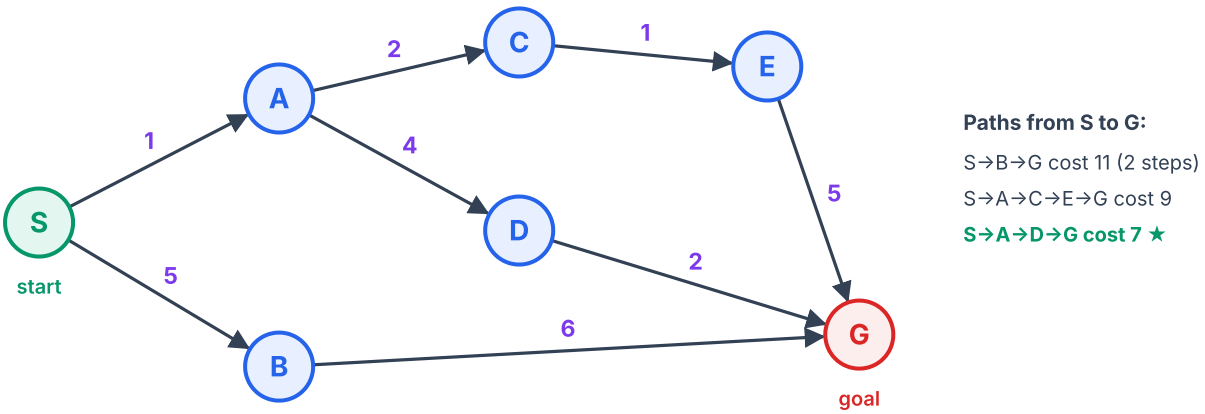


Fig 3.5 Our example graph for all uninformed searches: start S, goal G, one-way roads with step costs.

CONVENTIONS FOR EVERY TRACE BELOW

- Children are generated in **alphabetical order**; ties in a priority queue are broken alphabetically.
- We use **graph search** (a state already expanded or already in the frontier is not added again, except UCS may *replace* a frontier entry with a cheaper one).
- **BFS** tests for the goal when a node is **generated** (textbook version); all other strategies test when a node is **popped** for expansion.

(1) Breadth-first search (BFS)

Idea: expand the shallowest unexpanded node first — the root, then all its children, then all their children … level by level, like ripples on water. **Frontier:** a **FIFO queue** (new nodes go to the back).

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)              /* the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

Step	Pop & expand	Children generated (goal-tested now)	Frontier after (front → back)	Explored
0	—	S is not the goal	[S]	{ }
1	S	A, B	[A, B]	{S}
2	A	C, D	[B, C, D]	{S, A}
3	B	**G — goal test succeeds ✔**	stop	{S, A, B}

Result: path **S → B → G**, cost 5 + 6 = **11**, depth 2. BFS found the *shallowest* goal (fewest steps) but not the cheapest (cost 7 exists).

Properties. Complete if b is finite (it reaches every depth). **Optimal** only if path cost is a non-decreasing function of depth, e.g. **all step costs equal**. **Time:** nodes generated $b + b^2 + \ldots + b^d = O(b^d)$ (it would be $O(b^{d+1})$ if the goal test were applied on expansion). **Space:** $O(b^d)$ — every generated node stays in memory ($O(b^{d-1})$ explored + $O(b^d)$ frontier).

Depth d	Nodes	Time	Memory
2	110	0.11 ms	107 KB
6	10^6	1.1 s	1 GB
10	10^{10}	3 hours	10 TB
14	10^{14}	3.5 years	99 PB
16	10^{16}	350 years	10 EB

BFS cost with b = 10, 1 million nodes/second, 1000 bytes/node (textbook figures). Two lessons: **memory is a bigger problem than time**, and exponential problems can't be solved by uninformed search except for small instances.

(2) Uniform-cost search (UCS)

Idea: expand the node with the lowest path cost $g(n)$ first — like BFS, but measuring distance in cost instead of steps (Dijkstra's idea). **Frontier:** a **priority queue ordered by g**. Two changes from BFS:

1. The goal test is applied when a node is **selected for expansion**, not when generated (the first goal generated may be on a sub-optimal path).
2. If a **cheaper path** to a node already in the frontier is found, the frontier node is **replaced**.

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)                /* the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Step	Pop (*g*)	Goal?	Children added (*g* = cost so far)	Frontier after (lowest *g* first)
0	—			{S:0}
1	S:0	no	A:1, B:5	{A:1, B:5}
2	A:1	no	C:1+2=3, D:1+4=5	{C:3, B:5, D:5}
3	C:3	no	E:3+1=4	{E:4, B:5, D:5}
4	E:4	no	G:4+5=9 (generated, **not** returned yet)	{B:5, D:5, G:9}
5	B:5	no	G:5+6=11 — worse than G:9 already in frontier ⇒ ignore	{D:5, G:9}
6	D:5	no	G:5+2=7 — **cheaper than G:9 ⇒ replace**	{G:7}
7	G:7	**YES ✔**	stop	

Result: S → A → D → G, cost 1 + 4 + 2 = **7** — the optimal path. Had we tested G when it was first generated (step 4), we would have returned the cost-9 path.

Properties. Complete if every step cost ≥ ε for some small positive ε (zero-cost loops such as repeated *NoOp* could otherwise trap it). **Optimal**: yes — nodes are expanded in order of their optimal path cost. **Time and space:** $O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$, where C^* = cost of the optimal solution; this can be much larger than b^d because UCS may explore large trees of cheap steps first. With all step costs equal it becomes $O(b^{d+1})$.

(3) Depth-first search (DFS)

Idea: expand the deepest unexpanded node first — go down one branch as far as possible, then back up (*backtrack*) to the next branch. Like exploring a maze by always taking the first unexplored corridor. **Frontier:** a **LIFO queue (stack)**: the most recently generated node is chosen first.

Step	Pop	Goal?	Push	Stack after (top first)
0				[S]
1	S	no	A, B	[A, B]
2	A	no	C, D	[C, D, B]
3	C	no	E	[E, D, B]
4	E	no	G	[G, D, B]
5	G	**YES ✔**		stop

Result: S → A → C → E → G, cost 1+2+1+5 = **9**, depth 4. Neither the cheapest (7) nor the shallowest (2) — DFS simply returns the *first* goal it bumps into.

Implementation: GRAPH-SEARCH with a LIFO queue, or (more commonly) a recursive function that calls itself on each child in turn.

Properties. Complete? The graph-search version is complete in *finite* state spaces; the tree-search version is **not** (it can loop forever, e.g. Arad–Sibiu–Arad–Sibiu …); both fail in infinite state spaces (they can go down an infinite path that never reaches the goal). **Optimal?** No. **Time:** $O(b^m)$ — terrible if $m >> d$. **Space:** only $O(bm)$ for tree search — it stores just the current path plus the unexpanded siblings of each node on it. This is DFS's big advantage: at d = 16 it needs about 156 KB instead of BFS's 10 exabytes. (**Backtracking search**, which generates one successor at a time, needs only $O(m)$.)

(4) Depth-limited search (DLS)

Idea: DFS with a **depth limit *l***: nodes at depth l are treated as if they have no successors. This removes the infinite-path problem. DFS is the special case $l = \infty$.

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
    cutoff_occurred? ← false
    for each action in problem.ACTIONS(node.STATE) do
      child  ← CHILD-NODE(problem, node, action)
      result ← RECURSIVE-DLS(child, problem, limit − 1)
      if result = cutoff then cutoff_occurred? ← true
      else if result ≠ failure then return result
    if cutoff_occurred? then return cutoff else return failure
```

```
l = 1:  S(d0) → A(d1, limit reached: cutoff) → B(d1, cutoff)              ⇒ returns CUTOFF
(no answer within depth 1)
l = 2:  S(d0) → A(d1) → C(d2, cutoff) → D(d2, cutoff) → B(d1) → G(d2) ✔  ⇒ returns S→B→G
(cost 11)
```

Two kinds of failure: *failure* = no solution at all; *cutoff* = no solution *within the depth limit*. **Properties:** **complete only if *l* ≥ *d*** (incomplete if $l < d$); **not optimal** (if $l > d$ it may return a deeper, costlier goal); time $O(b^l)$; space $O(bl)$. **Choosing *l*:** use problem knowledge — Romania has 20 cities, so l = 19 is safe; better, any city is reachable from any other in at most 9 steps (the **diameter** of the state space), so l = 9.

(5) Iterative deepening search (IDS)

Idea: run DLS with l = 0, 1, 2, 3, … until a goal is found. It combines the small memory of DFS with the completeness (and shallowest-goal optimality) of BFS.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  for depth = 0 to ∞ do
    result ← DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result
```

Limit *l*	Nodes visited, in order	Result of this iteration
0	S	cutoff
1	S, A, B	cutoff
2	S, A, C, D, B, **G**	**found S → B → G (depth 2, cost 11)**

"Isn't repeating the top levels wasteful?" Not much: most nodes are at the bottom level. Nodes at depth d are generated once, depth d−1 twice, …, depth 1 d times:

$$N(\text{IDS}) = (d)b + (d-1)b^2 + \ldots + (1)b^d = O(b^d)$$

b = 10, d = 5: N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450 vs N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110 (only about 11% more)

Properties: complete (if b finite); optimal if all step costs are equal; time $O(b^d)$; space $O(bd)$. IDS is the preferred uninformed method when the search space is large and the depth of the solution is not known.

(6) Bidirectional search

Idea: run **two simultaneous searches** — one **forward** from the initial state and one **backward** from the goal — and stop when the two frontiers **meet**. Like two tunnel crews digging from both sides of a mountain. Motivation: $b^{d/2} + b^{d/2}$ is much less than b^d. E.g. b = 10, d = 6: about 2,220 nodes instead of 1,111,110 for BFS.

Round	Forward search from S	Backward search from G (follow arrows in reverse)	Frontiers intersect?
0	{S}	{G}	no
1	expand S → {A, B}	expand G → predecessors {B, D, E}	**yes, at B ✔**

Result: S → B (forward half) + B → G (backward half) = **S → B → G**. Each search went only 1 level instead of 2. **Properties:** complete and optimal (for equal step costs) if both directions use BFS; time and space $O(b^{d/2})$. **Limitations:** (i) we must be able to compute **predecessors** (reverse actions); (ii) the goal must be an explicit state — hard for abstract goals like "checkmate"; (iii) one frontier must be kept in memory to check intersections, so space is the weak point. The goal test is replaced by "do the frontiers intersect?".

Comparison of uninformed strategies ★ Very likely

Criterion	BFS	Uniform-cost	DFS	Depth-limited	Iterative deepening	Bidirectional
Frontier	FIFO queue	priority queue (g)	LIFO stack	LIFO stack	LIFO stack	two queues
Complete?	Yes[a]	Yes[a,b]	No	No (yes if $l \geq d$)	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]
On our graph	S-B-G (11)	S-A-D-G (7)	S-A-C-E-G (9)	l=2: S-B-G (11)	S-B-G (11)	S-B-G (11)

[a] complete if b is finite; [b] complete if step costs ≥ ε for positive ε; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search. (Tree-search versions; b = branching factor, d = depth of shallowest solution, m = maximum depth, l = depth limit, C^* = optimal cost.)

EXAM TIP — HOW TO WRITE A SEARCH TRACE FOR FULL MARKS

- Draw the graph first, then a table with columns **Step | Node expanded | Frontier | Explored** (add *g* or *f* values for UCS/A*).
- State the data structure (**FIFO / LIFO / priority queue**) and your tie-breaking rule (alphabetical).
- Finish with the **path**, its **cost**, and a one-line comment ("BFS found the shallowest, not the cheapest, path").
- For a "compare" question, reproduce the comparison table with footnotes — examiners check the complexities.

COMMON MISTAKES

- **"BFS is always optimal."** Only when all step costs are equal; otherwise use UCS.
- **"DFS is complete."** Not in infinite spaces (or tree search with loops). DFS is never optimal.
- Doing the goal test on **generation** in UCS/A* — it must be done when the node is **popped**.
- Writing DFS space as $O(b^m)$ — that is its *time*; space is $O(bm)$.
- Confusing *d* (depth of the shallowest goal) with *m* (maximum depth of the space).

3.5 Informed (heuristic) search strategies

Informed search uses problem-specific knowledge beyond the problem definition to find solutions more efficiently. The general approach is **best-first search**: a graph/tree search in which the node selected for expansion is the one with the lowest value of an **evaluation function *f*(*n*)** (a cost estimate). It is identical to UCS except that *f* replaces *g* in the priority queue. Most best-first algorithms include a heuristic function in *f*.

DEFINITION — HEURISTIC FUNCTION

$h(n)$ = estimated cost of the cheapest path from the state at node *n* to a goal state. It depends only on the state, is non-negative, and $h(n) = 0$ if *n* is a goal. *Example:* h_{SLD} = straight-line distance to Bucharest (you can't compute it from the problem description; it is extra knowledge).

Greedy best-first search

Idea: expand the node that appears closest to the goal: $f(n) = h(n)$. "Always walk towards the city that looks nearest to Bucharest as the crow flies."

Step	Expand (*h*)	Children added (*h*)	Frontier after (lowest *h* first)
1	Arad 366	Sibiu 253, Timisoara 329, Zerind 374	Sibiu 253, Timisoara 329, Zerind 374
2	Sibiu 253	Arad 366, Fagaras 176, Oradea 380, Rimnicu Vilcea 193	Fagaras 176, R.Vilcea 193, Timisoara 329, Arad 366, Zerind 374, Oradea 380
3	Fagaras 176	Sibiu 253, Bucharest 0	Bucharest 0, R.Vilcea 193, Sibiu 253, Timisoara 329, …
4	Bucharest 0	**goal ✔** Path Arad → Sibiu → Fagaras → Bucharest = 140 + 99 + 211 = **450 km**	

Very fast (it never expanded a node off the solution path) but not optimal: the route via Rimnicu Vilcea and Pitesti is 418 km, 32 km shorter. **Properties: not optimal**; the tree-search version is **incomplete** even in a finite space (from Iasi to Fagaras it goes to the dead end Neamt, back to Iasi, Neamt again … forever); the graph-search version is complete in finite spaces. Worst-case time and space $O(b^m)$, but a good heuristic reduces this dramatically.

A* search: minimizing the total estimated solution cost ★ Very likely

$$f(n) = g(n) + h(n)$$

$g(n)$ = cost so far to reach n · $h(n)$ = estimated cost from n to the goal · $f(n)$ = estimated cost of the cheapest solution **through** n

A* is identical to uniform-cost search except that it orders the priority queue by $g + h$ instead of g. It combines UCS (exact cost so far, keeps it honest) with greedy search (estimate of the rest, keeps it focused).

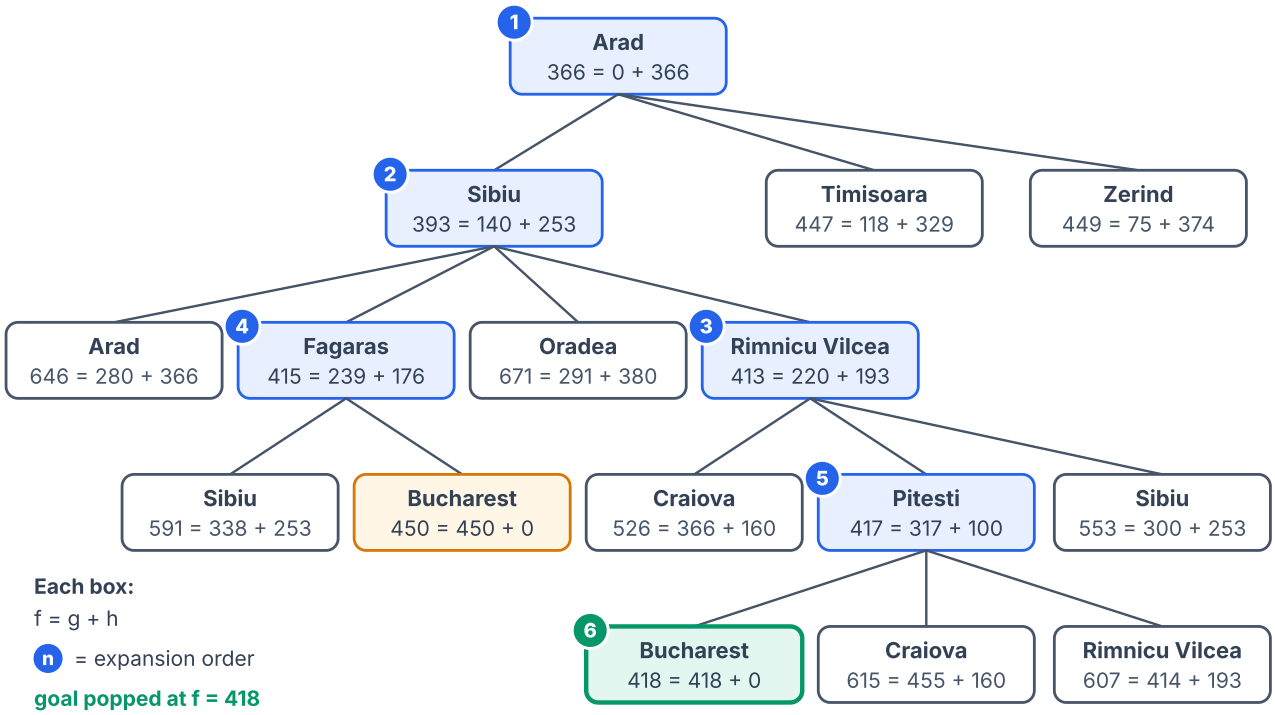


Fig 3.6 A search tree for Arad → Bucharest with h_{SLD}. Numbers in circles give the order of expansion.*

Step	Expand (*f*)	New nodes: *f* = *g* + *h*	Frontier after (lowest *f* first)
1	Arad 366	Sibiu 393 = 140+253; Timisoara 447 = 118+329; Zerind 449 = 75+374	Sibiu 393, Timisoara 447, Zerind 449
2	Sibiu 393	Arad 646 = 280+366; Fagaras 415 = 239+176; Oradea 671 = 291+380; R.Vilcea 413 = 220+193	R.Vilcea 413, Fagaras 415, Timisoara 447, Zerind 449, Arad 646, Oradea 671
3	R.Vilcea 413	Craiova 526 = 366+160; Pitesti 417 = 317+100; Sibiu 553 = 300+253	Fagaras 415, Pitesti 417, Timisoara 447, Zerind 449, Craiova 526, Sibiu 553, Arad 646, Oradea 671
4	Fagaras 415	Sibiu 591 = 338+253; **Bucharest 450 = 450+0** (goal generated, but not popped)	Pitesti 417, Timisoara 447, Zerind 449, Bucharest 450, Craiova 526, Sibiu 553, Sibiu 591, …
5	Pitesti 417	**Bucharest 418 = 418+0**; Craiova 615 = 455+160; R.Vilcea 607 = 414+193	Bucharest 418, Timisoara 447, Zerind 449, Bucharest 450, Craiova 526, …
6	**Pop Bucharest (418) — goal ✔** Path: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest = 140 + 80 + 97 + 101 = 418 km (optimal)		

Why didn't A* stop at step 4? Bucharest (450) was only *generated*; Pitesti had a lower *f* (417), meaning a cheaper route through Pitesti might exist — and it did. (This trace is the textbook's tree-search version; graph search would simply skip the repeated states Arad, Sibiu, R.Vilcea and give the same answer.) Note also that Timisoara (447) and Zerind (449) are **never expanded**: their *f* exceeds 418, so their whole subtrees are **pruned**.

Conditions for optimality: admissibility and consistency

ADMISSIBLE HEURISTIC

$h(n)$ is **admissible** if it never overestimates the true cost to reach the goal:

$$0 \le h(n) \le h^*(n) \text{ for every } n$$

(h^* = true optimal cost.) Admissible heuristics are **optimistic**. h_{SLD} is admissible because the straight line is the shortest path between two points, so it can never be an overestimate.

CONSISTENT (MONOTONE) HEURISTIC

For every node n and every successor n' generated by action a:

$$h(n) \le c(n, a, n') + h(n')$$

This is the **triangle inequality**: one side of the triangle ($n \to$ G) can't be longer than the sum of the other two. E.g. h(Sibiu) = 253 ≤ 80 + h(R.Vilcea) = 80 + 193 = 273 ✔.

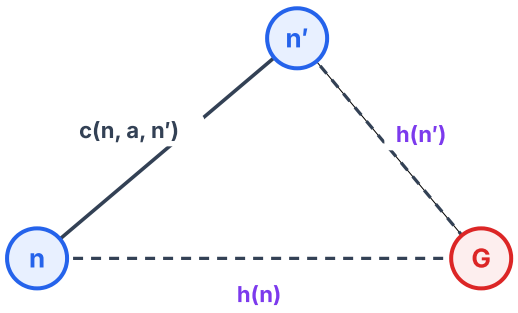


Fig 3.7 Triangle inequality behind consistency.

- Every consistent heuristic is also admissible (but a few admissible heuristics are not consistent). h_{SLD} is consistent.
- **Theorem:** the tree-search version of A* is optimal if h is admissible; the graph-search version is optimal if h is consistent.
- If h is consistent, the values of f along any path are **non-decreasing**: $f(n') = g(n) + c(n,a,n') + h(n') \ge g(n) + h(n) = f(n)$. So the sequence of nodes expanded by A* is in non-decreasing order of f, and the first goal selected for expansion must be optimal.

PROOF SKETCH (10M ANSWERS) — A* WITH AN ADMISSIBLE H IS OPTIMAL (TREE SEARCH)

Let C^* be the optimal cost. Suppose a sub-optimal goal G_2 (cost $C_2 > C^*$) is in the frontier. Let n be a frontier node on an optimal path (one always exists until the goal is found).

1. $f(G_2) = g(G_2) + 0 = C_2 > C^*$ (since h(goal) = 0).
2. $f(n) = g(n) + h(n) \le g(n) + h^*(n) = C^*$ (admissibility).
3. So $f(n) \le C^* < f(G_2)$: A* always expands n before G_2, so it never returns the sub-optimal goal. ∎

Other properties of A*. A* expands **all** nodes with $f(n) < C^*$, **some** nodes on the "goal contour" $f(n) = C^*$, and **no** nodes with $f(n) > C^*$ (think of contour lines on a map spreading out from the start). It is **complete** if there are finitely many nodes with $f \le C^*$ (true if step costs ≥ ε and b is finite), and **optimally efficient**: no other optimal algorithm using the same heuristic is guaranteed to expand fewer nodes. **Drawback:** the number of nodes is still exponential in the worst case, and because A* keeps all generated nodes in memory, it usually runs out of space long before it runs out of time.

Memory-bounded variant	Idea in one line
IDA* (iterative-deepening A*)	like IDS, but the cutoff is the *f*-cost (*g* + *h*) instead of depth; each iteration's cutoff = smallest *f* that exceeded the previous cutoff.
RBFS (recursive best-first search)	recursive, linear-space best-first search; keeps the *f*-value of the best alternative path (*f_limit*) and backtracks when the current node exceeds it, backing up the best child's *f*-value.
MA* / SMA* (simplified memory-bounded A*)	runs A* until memory is full, then drops the **worst** leaf (highest *f*) and backs its value up to its parent, so it can be regenerated later if needed.

COMMON MISTAKES

- **"Greedy best-first is optimal."** No — it ignores $g(n)$ (Arad→Bucharest: 450 km instead of 418).
- **An admissible *h* must never OVERestimate** (underestimating is fine). $h(n) = 0$ for all n is admissible (A* becomes UCS).
- Stopping A* when the goal is *generated*; it stops only when the goal is *popped* (step 6, not step 4).
- Arithmetic slips in *g*: always add the road length to the **parent's** *g*, e.g. *g*(Pitesti) = 220 + 97 = 317.

3.6 Heuristic functions ★ Very likely

The 8-puzzle facts: average solution cost of a random instance is about **22 steps**, branching factor about **3** (blank in the middle: 4 moves; on an edge: 3; in a corner: 2). Exhaustive tree search to depth 22 would examine about $3^{22} \approx 3.1 \times 10^{10}$ states; graph search cuts this to 181,440 distinct states (the 15-puzzle has about 10^{13}), so we need a good heuristic that **never overestimates**.

H_1 = NUMBER OF MISPLACED TILES

Count the tiles (not the blank) that are not in their goal position. **Admissible** because every misplaced tile must be moved at least once.

H_2 = SUM OF MANHATTAN DISTANCES

For each tile, add its horizontal + vertical distance from its goal square (tiles can't move diagonally) — also called **city-block distance**. **Admissible** because one move can bring only one tile one step closer.

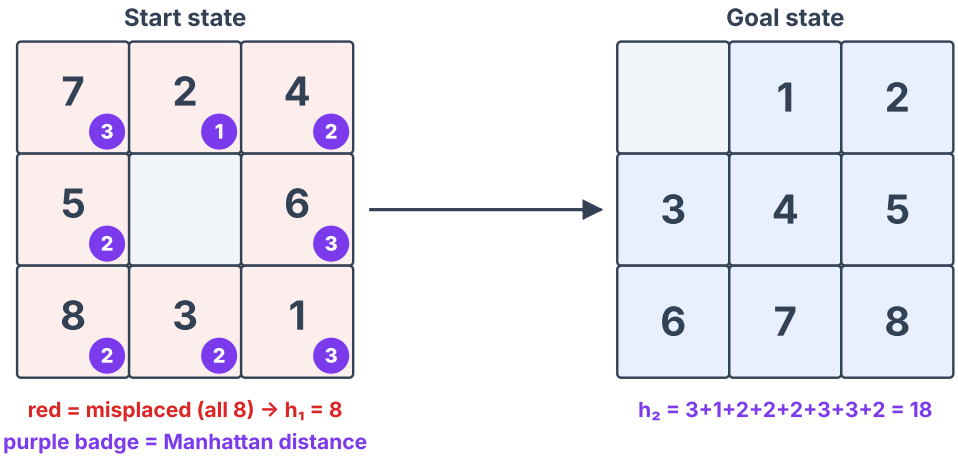


Fig 3.8 Computing h_1 and h_2 for the standard start state.

WORKED EXAMPLE — H_1 AND H_2 FOR START 7 2 4 / 5 _ 6 / 8 3 1 → GOAL _ 1 2 / 3 4 5 / 6 7 8

Tile	1	2	3	4	5	6	7	8	Total
Misplaced?	yes	yes	yes	yes	yes	yes	yes	yes	$h_1 = 8$
Manhattan distance	3	1	2	2	2	3	3	2	$h_2 = 18$

E.g. tile 1 is at row 3, column 3 and belongs at row 1, column 2: $|3-1| + |3-2| = 2 + 1 = 3$. Tile 7 is at row 1, column 1 and belongs at row 3, column 2: $2 + 1 = 3$. The true optimal solution cost is **26**, so both $8 \le 26$ and $18 \le 26$ are underestimates, as admissibility requires.

Effective branching factor b^* (quality of a heuristic)

If A* generates N nodes in total and the solution depth is d, then b^* is the branching factor that a **uniform tree** of depth d would need in order to contain $N + 1$ nodes:

$$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$$

Easy example: $d = 2$, $N = 6 \Rightarrow 7 = 1 + b^* + (b^*)^2 \Rightarrow (b^*)^2 + b^* - 6 = 0 \Rightarrow (b^* + 3)(b^* - 2) = 0 \Rightarrow$ **$b^* = 2$**

Textbook example: solution at depth 5 using 52 nodes $\Rightarrow 53 = 1 + b^* + \ldots + (b^*)^5 \Rightarrow$ **$b^* \approx 1.92$** (check: $1 + 1.92 + 3.69 + 7.08 + 13.59 + 26.09 \approx 53.4$)

A well-designed heuristic has b^* close to 1, so fairly large problems can be solved. In the textbook's experiment at solution depth 12, IDS generated about 3.6 million nodes, A* with h_1 about 227 and A* with h_2 only about 73.

Dominance

If $h_2(n) \ge h_1(n)$ for every node n (both admissible), we say **h_2 dominates h_1**. Then A* using h_2 will **never expand more nodes** than A* using h_1 (except possibly some nodes with $f = C^*$). Reason: A* surely expands every node with $f(n) < C^*$, i.e. with $h(n) < C^* - g(n)$; every node that satisfies this for the larger h_2 also satisfies it for h_1. So **it is always better to use the larger admissible heuristic**, provided it is cheap to compute. Manhattan distance dominates misplaced tiles (a misplaced tile is at distance ≥ 1).

Generating admissible heuristics from relaxed problems

A **relaxed problem** is the problem with **fewer restrictions on the actions**. Its state-space graph is a supergraph of the original (extra edges = shortcuts), so the cost of an optimal solution to a relaxed problem is an admissible (and consistent) heuristic for the original problem. 8-puzzle rule: "A tile can move from square A to square B if A is horizontally or vertically adjacent to B **and** B is blank." Removing conditions gives:

Relaxed rule	Exact cost of the relaxed problem
(a) A tile can move from A to B if A is adjacent to B	h_2 (Manhattan distance)
(b) A tile can move from A to B if B is blank	Gaschnig's heuristic
(c) A tile can move from A to B	h_1 (misplaced tiles)

If we have several admissible heuristics and none dominates, use $h(n) = \max\{h_1(n), \ldots, h_m(n)\}$ — still admissible, and it dominates all of them.

- **Pattern databases (subproblems):** store the *exact* solution cost of every configuration of a subproblem, e.g. getting tiles 1, 2, 3, 4 (and the blank) into place while ignoring the others. The cost is

a lower bound on the whole problem, so it is admissible. Built once by searching backward from the goal; lookups are cheap. **Disjoint** pattern databases (1-2-3-4 and 5-6-7-8, counting only moves of their own tiles) can be **added** and stay admissible.

- **Learning heuristics from experience:** solve many 8-puzzles optimally; each state on a solution path is an example (state, true cost). Learn h from **features**, e.g. $x_1(n)$ = number of misplaced tiles, $x_2(n)$ = number of pairs of adjacent tiles that are not adjacent in the goal: $h(n) = c_1 x_1(n) + c_2 x_2(n)$, with constants fitted to the data (not guaranteed admissible).

Uninformed vs informed search

	Uninformed (blind) search	Informed (heuristic) search
Knowledge	only the problem definition	problem definition + heuristic $h(n)$
Chooses node by	depth (BFS/DFS) or path cost $g(n)$ (UCS)	$f(n) = h(n)$ (greedy) or $g(n) + h(n)$ (A*)
Efficiency	explores many useless nodes; slow for big problems	focuses on promising nodes; far fewer nodes
Optimal algorithms	UCS; BFS/IDS for equal step costs	A* with admissible (tree) / consistent (graph) h
Examples	BFS, UCS, DFS, DLS, IDS, bidirectional	greedy best-first, A*, IDA*, RBFS, SMA*
Extra cost	none	designing and computing h

EXAM TIP — A* / HEURISTIC QUESTIONS (10M)

- Write $f(n) = g(n) + h(n)$ and define each term. Then do the Romania trace as a table with "$f = g + h$" for every node (show the additions!) and draw the tree (Fig 3.6).
- Always end with the **path and cost (418 km)** and the comparison with greedy (450 km).
- For "admissible vs consistent": two definitions + formula + triangle figure + the optimality theorem.
- For 8-puzzle heuristics: draw both boards, compute h_1 and h_2 tile by tile, then mention dominance and b^*.

LIKELY EXAM QUESTIONS — CHAPTER 3

1. What are the components of a well-defined problem? Formulate the vacuum world / 8-puzzle / 8-queens / Romania route-finding problem. **(6–8M)**
2. Differentiate tree search and graph search. Explain frontier, explored set and the node data structure. **(6M)**
3. Explain BFS with algorithm and example; analyse its time and space complexity. **(8M)**
4. Explain uniform-cost search. Why is the goal test applied on expansion? Trace it on a given graph. **(8M)**
5. Explain DFS, depth-limited search and iterative deepening search with examples. **(10M)**
6. Compare all uninformed search strategies (completeness, optimality, time, space). **(6M)**
7. Explain greedy best-first search and A* on the Romania map. Why is greedy not optimal? **(10M)**
8. Define admissible and consistent heuristics. Prove that A* is optimal when h is admissible. **(8M)**
9. For the given 8-puzzle state compute h_1 and h_2. What is dominance? What is the effective branching factor? **(6M)**
10. How can admissible heuristics be generated from relaxed problems? **(4M)**

60-SECOND RECAP

- Problem = **initial state, actions, transition model, goal test, path cost**. Optimal solution = lowest path cost.
- Frontier = open list of leaf nodes; explored set = closed list (graph search avoids repeated states). Measure with completeness, optimality, time, space using b, d, m.
- **BFS** FIFO, $O(b^d)$ time & space, optimal for equal costs · **UCS** lowest g, optimal · **DFS** LIFO, $O(bm)$ space, not complete/optimal · **DLS** depth limit l · **IDS** best of BFS + DFS, $O(bd)$ space · **Bidirectional** $O(b^{d/2})$.
- **Greedy**: $f = h$, fast, not optimal (450 km). **A***: $f = g + h$, optimal with admissible (tree) / consistent (graph) h (418 km).
- 8-puzzle: h_1 = misplaced tiles (8), h_2 = Manhattan (18); h_2 dominates h_1; good heuristics have b^* near 1; relaxed problems give admissible heuristics.

4 Logical Agents & Propositional Logic

Humans know things and reason with that knowledge. A **knowledge-based agent** keeps facts as sentences in a logic and *derives* new facts to decide what to do. We use the Wumpus world as the running example and propositional logic as the language.

4.1 Knowledge-based agents

DEFINITIONS

- **Knowledge base (KB)** — a set of sentences that represent what the agent knows about the world. (Think: the agent's notebook of facts and rules.)
- **Sentence** — an assertion about the world, expressed in a **knowledge representation language**. A sentence taken as given without being derived from other sentences is an **axiom**.
- **TELL** — adds a new sentence to the KB. **ASK** — queries what is known.
- **Inference** — deriving new sentences from old ones. The answer to an ASK must *follow* from what has been TELLed; inference should not make things up.
- **Background knowledge** — what the KB contains initially, before any percepts.

```
function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
              t, a counter, initially 0, indicating time
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

Each time it is called the agent does three things: (1) **TELLs** the KB what it perceives, (2) **ASKs** the KB what action to perform (extensive reasoning about the current state, outcomes of actions, etc. happens here), (3) **TELLs** the KB which action was chosen, and executes it.

Function	What it does
`MAKE-PERCEPT-SENTENCE`	builds a sentence asserting that the agent perceived the given percept at time *t*
`MAKE-ACTION-QUERY`	builds a sentence that asks what action should be done at time *t*
`MAKE-ACTION-SENTENCE`	builds a sentence asserting that the chosen action was executed

Knowledge level vs implementation level

At the **knowledge level** we describe an agent only by *what it knows and what its goals are* (e.g. a taxi knows that a bridge links two parts of the city and its goal is to take a passenger across). At the **implementation level** we describe the *data structures and algorithms* that store and manipulate that knowledge. We can design at the knowledge level without worrying about implementation.

Declarative vs procedural approach

Declarative: build the agent by TELLing it sentences one by one until it knows how to operate (state *what* is true). **Procedural:** encode the desired behaviours directly as program code (state *how* to act). Successful agents usually combine both, and add learning.

4.2 The Wumpus world ★ Very likely

A cave of rooms (a 4×4 grid) connected by passageways. Somewhere lurks the terrible **wumpus**, a beast that eats anyone who enters its room. The agent has **one arrow** to shoot it. Some rooms contain bottomless **pits** that trap anyone who wanders in (the wumpus is too big to fall in). One room contains a heap of **gold**. The agent must find the gold and get out alive.

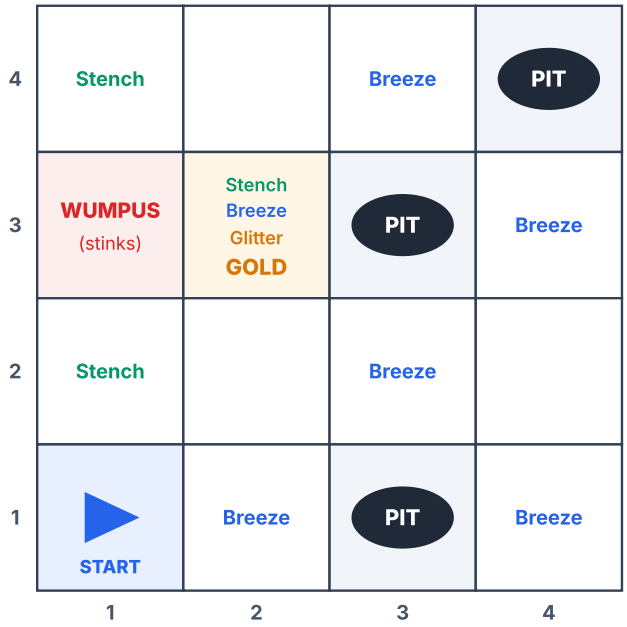


Fig 4.1 A typical Wumpus world. The agent starts at [1,1] facing right.

PEAS description	
P	**+1000** for climbing out of the cave with the gold; **−1000** for falling into a pit or being eaten by the wumpus; **−1** for each action taken; **−10** for using up the arrow. The game ends when the agent dies or climbs out.
E	4×4 grid of rooms. Agent starts in [1,1] facing east (right). Gold and wumpus are placed randomly in squares other than [1,1]; every other square is a pit with probability 0.2.
A	*Forward*, *TurnLeft*, *TurnRight* (by 90°), *Grab* (pick up gold in the same square), *Shoot* (arrow flies straight ahead until it kills the wumpus or hits a wall; only the first Shoot has an effect), *Climb* (leave the cave, only from [1,1]).
S	5 sensors, each giving one bit: **Stench** (in the wumpus's square and directly adjacent ones, not diagonal), **Breeze** (adjacent to a pit), **Glitter** (in the gold's square), **Bump** (walked into a wall), **Scream** (wumpus killed; heard anywhere).

The percept is a **5-tuple** [*Stench, Breeze, Glitter, Bump, Scream*]. E.g. stench and breeze but nothing else: `[Stench, Breeze, None, None, None]`. Entering a square with a pit or a live wumpus kills the agent (a dead wumpus is safe but still smells).

Environment properties:

- **Partially observable** — only local perception (you smell/feel only neighbours).
- **Deterministic** — outcomes are exactly specified.
- **Sequential** — rewards may come only after many actions.
- **Static** — the wumpus and pits don't move.
- **Discrete** — squares, actions and percepts are discrete.
- **Single-agent** — the wumpus is a natural feature, not an agent.
- Rules ("physics") **known**; the layout is **unknown** to the agent.

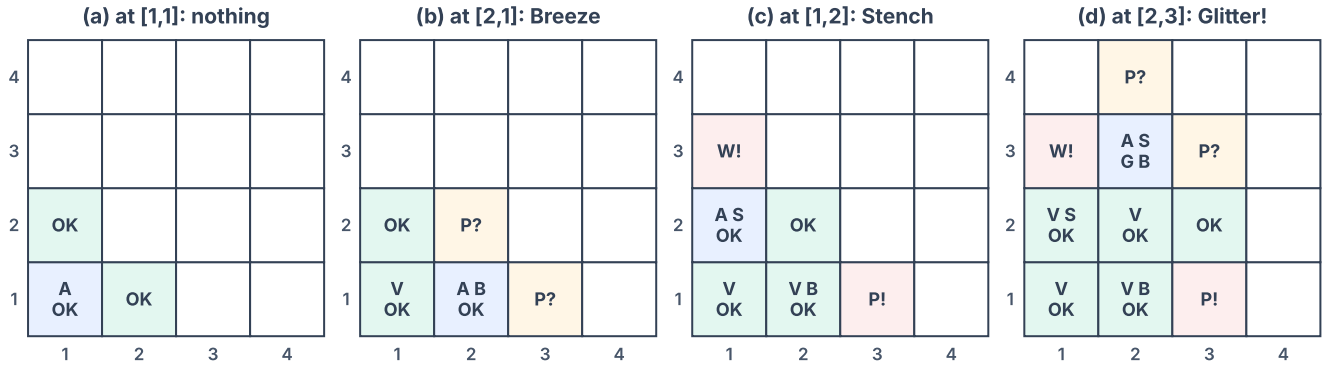
MEMORY TRICKS

Sensors: "**S**ome **B**rave **G**uys **B**ump into **S**creams" → Stench, Breeze, Glitter, Bump, Scream (in tuple order).

Scores: "gold out +1000, death −1000, arrow −10, every step −1".

Clues: **S**tench = **S**omething (wumpus) next door; **B**reeze = **B**ottomless pit next door.

Step-by-step reasoning of the agent



A = agent · B = breeze · G = glitter · OK = safe · P = pit · S = stench · V = visited · W = wumpus · ? = maybe · ! = certain

Fig 4.2 What the agent knows after each move (for the world in Fig 4.1).

1. **At [1,1]:** percept `[None, None, None, None, None]`. No breeze ⇒ no pit in the neighbours; no stench ⇒ no wumpus in the neighbours. So [1,2] and [2,1] are OK (safe). The agent moves to [2,1].
2. **At [2,1]:** percept `[None, Breeze, None, None, None]`. Breeze ⇒ a pit in [2,2] or [3,1] (marked P?). [1,1] is already known safe. Being cautious, the agent turns back to [1,1] and then goes to the other safe square [1,2].
3. **At [1,2]:** percept `[Stench, None, None, None, None]`. Stench ⇒ the wumpus is in [1,3] or [2,2]. But there was **no stench in [2,1]**, so the wumpus is not in [2,2] ⇒ wumpus in [1,3] (W!). No breeze in [1,2] ⇒ no pit in [2,2] (or [1,3]); combined with the breeze in [2,1] ⇒ pit in [3,1] (P!). Now [2,2] is OK.
4. **At [2,2]:** nothing perceived ⇒ [3,2] and [2,3] are OK. The agent moves to [2,3].
5. **At [2,3]:** `[Stench, Breeze, Glitter, None, None]` ⇒ **Grab** the gold, return along safe squares to [1,1] and **Climb** out. (The breeze here means a pit in [2,4] or [3,3] — not yet certain.)

KEY POINT

In each case where the agent draws a conclusion from the available information, that conclusion is guaranteed to be correct if the available information is correct. This is the fundamental property of logical reasoning.

4.3 Logic: syntax, semantics, entailment ★ Very likely

Syntax the rules for **well-formed sentences**. "$x + y = 4$" is well formed; "$x4y+ =$" is not.

Semantics the **meaning** of sentences: defines the **truth** of each sentence with respect to each **possible world**. "$x + y = 4$" is true in a world where $x = 2$, $y = 2$, false where $x = 1$, $y = 1$.

Model a mathematical abstraction of a possible world: it fixes the truth or falsehood of every relevant sentence. If sentence α is true in model m, we say ***m* satisfies α** (or m is a model of α). **M(α)** = the set of all models of α.

ENTAILMENT

α ⊨ β ("α entails β", "β logically follows from α") means: in every model in which α is true, β is also true.

$$\alpha \models \beta \text{ if and only if } M(\alpha) \subseteq M(\beta)$$

Example: ($x = 0$) ⊨ ($xy = 0$). Note: if α ⊨ β, α is the *stronger* assertion — it rules out more possible worlds.

WORKED EXAMPLE — ENTAILMENT BY MODEL CHECKING IN THE WUMPUS WORLD

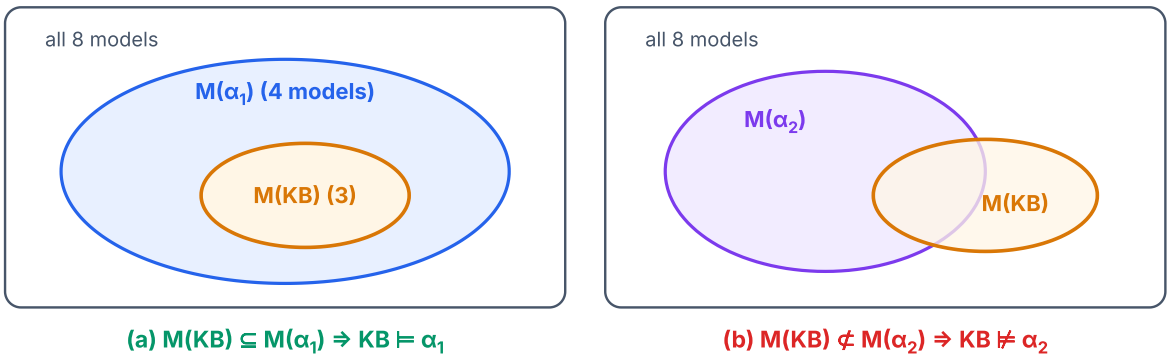
The agent has felt nothing in [1,1] and a breeze in [2,1]. KB = these percepts + the rules of the world. Are there pits in [1,2], [2,2], [3,1]? Three yes/no symbols ⇒ 2^3 = 8 possible models.

$P_{1,2}$	$P_{2,2}$	$P_{3,1}$	KB?	$\alpha_1 = \neg P_{1,2}$	$\alpha_2 = \neg P_{2,2}$
F	F	F	F	T	T
F	F	T	T	T	T
F	T	F	T	T	F
F	T	T	T	T	F
T	F	F	F	F	T
T	F	T	F	F	T
T	T	F	F	F	F
T	T	T	F	F	F

KB is false when $P_{1,2}$ is true (there would be a breeze in [1,1]) and when neither [2,2] nor [3,1] has a pit (no breeze in [2,1]). So KB is true in exactly **3 models**.

α_1 "no pit in [1,2]" is true in all 3 ⇒ $KB \models \alpha_1$.

α_2 "no pit in [2,2]" is false in 2 of them ⇒ **$KB \not\models \alpha_2$**: the agent cannot conclude that [2,2] is pit-free (or that it has one).



(a) $M(KB) \subseteq M(\alpha_1) \Rightarrow KB \models \alpha_1$ (b) $M(KB) \not\subseteq M(\alpha_2) \Rightarrow KB \not\models \alpha_2$

Fig 4.3 Entailment as a subset relation between sets of models.

Inference, soundness, completeness, grounding

Logical inference derives conclusions: $KB \vdash_i \alpha$ means "α is derived from KB by inference algorithm *i*". **Model checking** is one inference algorithm: enumerate all possible models and check that α is true in every model in which KB is true. *Analogy:* the set of all consequences of KB is a haystack and α a needle; **entailment** = the needle is in the haystack; **inference** = finding it.

SOUND (TRUTH-PRESERVING)

An inference algorithm is **sound** if it derives only entailed sentences: whenever $KB \vdash_i \alpha$, also $KB \models \alpha$. An unsound procedure "makes things up" (announces needles that aren't there).

COMPLETE

An inference algorithm is **complete** if it can derive any sentence that is entailed: whenever $KB \models \alpha$, also $KB \vdash_i \alpha$. It never misses a needle.

With a sound inference procedure, **if KB is true in the real world, then any sentence α derived from KB is also true in the real world**. **Grounding** asks how we know KB is true in the real world at all: the connection is the agent's **sensors** (percept sentences are true because sensors produced them) and **learning** (general rules such as "squares next to pits are breezy" come from experience; learning can be wrong, so this is not guaranteed).

MEMORY TRICK

⊨ (double bar) = **semantics** — "is *truly* implied in all models". ⊢ (single bar) = **syntax** — "can be *derived* by the algorithm". **Sound** = "says nothing false"; **Complete** = "misses nothing true".

4.4 Propositional logic: a very simple logic ★ Very likely

Syntax

- **Atomic sentence** = a single **proposition symbol** standing for a proposition that can be true or false: *P*, *Q*, *R*, $W_{1,3}$ ("the wumpus is in [1,3]"), *North*. Two symbols have fixed meanings: **True** (always true) and **False** (always false).
- **Complex sentences** are built from simpler ones with parentheses and the five **logical connectives**:

Symbol	Name	Read as	Example and parts
¬	**Negation**	not	$\neg W_{1,3}$. A **literal** is an atomic sentence (*positive literal*) or a negated atomic sentence (*negative literal*).
∧	**Conjunction**	and	$W_{1,3} \wedge P_{3,1}$; the parts are **conjuncts**.
∨	**Disjunction**	or	$(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$; the parts are **disjuncts**.
⇒	**Implication** (conditional)	implies, if…then	$(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$. Left part = **premise / antecedent**, right part = **conclusion / consequent**. Also called *rules* or if–then statements (sometimes written ⊃ or →).
⇔	**Biconditional**	if and only if	$W_{1,3} \Leftrightarrow \neg W_{2,2}$ (sometimes written ≡).

```
Sentence        → AtomicSentence |
ComplexSentence
AtomicSentence  → True | False | P | Q | R |
...
ComplexSentence → ( Sentence ) | [ Sentence ]
                |  ¬ Sentence
                |  Sentence ∧ Sentence
                |  Sentence ∨ Sentence
                |  Sentence ⇒ Sentence
                |  Sentence ⇔ Sentence
OPERATOR PRECEDENCE : ¬, ∧, ∨, ⇒, ⇔
```

BNF grammar of propositional logic (left). **Precedence** from highest to lowest: ¬ , ∧ , ∨ , ⇒ , ⇔ . So

$$\neg A \wedge B \Rightarrow C \text{ means } ((\neg A) \wedge B) \Rightarrow C$$

MEMORY TRICK — PRECEDENCE

"**N**aughty **A**nts **O**ften **I**nvade **B**akeries" → **N**ot, **A**nd, **O**r, **I**mplies, **B**iconditional.

Semantics and the truth table

A **model** in propositional logic simply fixes the truth value (true/false) of every proposition symbol, e.g. $m_1 = \{P_{1,2} = false, P_{2,2} = false, P_{3,1} = true\}$. With *n* symbols there are 2^n models. The truth of a complex sentence is computed recursively with these rules:

P	*Q*	¬*P*	*P* ∧ *Q*	*P* ∨ *Q*	*P* ⇒ *Q*	*P* ⇔ *Q*
false	false	true	false	false	**true**	true
false	true	true	false	true	**true**	false
true	false	false	false	true	**false**	false
true	true	false	true	true	**true**	true

- ¬*P* is true iff *P* is false.
- *P* ∧ *Q* is true iff both are true.
- *P* ∨ *Q* is true iff at least one is true.
- *P* ⇒ *Q* is true **unless *P* is true and *Q* is false**.
- *P* ⇔ *Q* is true iff *P* and *Q* are both true or both false.

COMMON MISTAKE — IMPLICATION

$P \Rightarrow Q$ is TRUE whenever P is false, whatever Q is. Read it as "if P is true, then I claim Q is true; otherwise I make no claim." So "5 is odd ⇒ Tokyo is the capital of Japan" is true, and "5 is even ⇒ Sam is smart" is true regardless of Sam. Implication does not need any causal relation between P and Q.

Evaluating a sentence: in m_1, $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = true \wedge (false \vee true) = true \wedge true = true$.

A simple knowledge base for the Wumpus world

Symbols for each [x,y]: $P_{x,y}$ = pit, $W_{x,y}$ = wumpus, $B_{x,y}$ = breeze, $S_{x,y}$ = stench (true if present).

Rule	Sentence	Meaning
R_1	$\neg P_{1,1}$	There is no pit in [1,1].
R_2	$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$	A square is breezy **if and only if** there is a pit in a neighbouring square.
R_3	$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$	Same rule for [2,1].
R_4	$\neg B_{1,1}$	Percept: no breeze in [1,1].
R_5	$B_{2,1}$	Percept: breeze in [2,1].

Why ⇔ and not ⇒? $B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})$ is true but incomplete: it does not say that a pit next door *causes* a breeze. We need both directions.

Inference by truth-table enumeration (model checking)

```
function TT-ENTAILS?(KB, α) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          α, the query, a sentence in propositional logic
  symbols ← a list of the proposition symbols in KB and α
  return TT-CHECK-ALL(KB, α, symbols, { })

function TT-CHECK-ALL(KB, α, symbols, model) returns true or false
  if EMPTY?(symbols) then
     if PL-TRUE?(KB, model) then return PL-TRUE?(α, model)
     else return true        // when KB is false, always return true
  else
     P ← FIRST(symbols);  rest ← REST(symbols)
     return (TT-CHECK-ALL(KB, α, rest, model ∪ {P = true})
             and TT-CHECK-ALL(KB, α, rest, model ∪ {P = false}))
```

The KB R_1–R_5 has 7 symbols ($B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, $P_{3,1}$) ⇒ 2^7 = **128 models**. KB is true in only **three** of them:

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1…R_5	KB
F	T	F	**F**	F	F	T	all true	**true**
F	T	F	**F**	F	T	F	all true	**true**
F	T	F	**F**	F	T	T	all true	**true**

$P_{1,2}$ is false in all three ⇒ $KB \models \neg P_{1,2}$ (no pit in [1,2]). $P_{2,2}$ is true in two and false in one ⇒ we cannot yet tell. The algorithm is **sound and complete**, but with n symbols its time is $O(2^n)$ (space $O(n)$, since it enumerates depth-first). Propositional entailment is co-NP-complete, so every known algorithm is exponential in the worst case.

4.5 Reasoning patterns: propositional theorem proving ★ Very likely

Instead of enumerating models, we can apply **inference rules** directly to sentences to build a **proof**. First, some concepts:

- **Logical equivalence:** $\alpha \equiv \beta$ if they are true in the same set of models. Equivalently, $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$.
- **Validity:** a sentence is **valid** (a **tautology**) if it is true in *all* models, e.g. $P \vee \neg P$. **Deduction theorem:** $\alpha \models \beta$ iff $(\alpha \Rightarrow \beta)$ is valid.
- **Satisfiability:** a sentence is **satisfiable** if it is true in *some* model (the **SAT** problem, the first problem proved NP-complete). α is valid iff $\neg\alpha$ is unsatisfiable.
- **Proof by contradiction (refutation, reductio ad absurdum):** $\alpha \models \beta$ iff $(\alpha \wedge \neg\beta)$ is unsatisfiable. Assume β is false and show this leads to a contradiction. This is the basis of resolution.

Equivalence	Name
$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha) \equiv \alpha$	double-negation elimination
$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	contraposition
$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	**implication elimination**
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	**biconditional elimination**
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	**De Morgan**
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	**De Morgan**
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	**distributivity of $\vee$ over $\wedge$**

Inference rules

Modus Ponens $\dfrac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$

If (WumpusAhead ∧ WumpusAlive) ⇒ Shoot and (WumpusAhead ∧ WumpusAlive) are given, infer Shoot.

And-Elimination $\dfrac{\alpha \wedge \beta}{\alpha}$

From (WumpusAhead ∧ WumpusAlive), infer WumpusAlive.

All the equivalences above can also be used as inference rules, e.g. biconditional elimination gives $\dfrac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$ and $\dfrac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$. These rules are **sound** (a line under premises means: whenever the premises are true, the conclusion is true).

WORKED PROOF — PROVE $\neg P_{1,2}$ (NO PIT IN [1,2]) FROM R_1–R_5

Step	Sentence derived	Rule used
R_6	$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$	biconditional elimination on R_2
R_7	$(P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}$	And-Elimination on R_6
R_8	$\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})$	contraposition of R_7
R_9	$\neg(P_{1,2} \vee P_{2,1})$	Modus Ponens with R_8 and R_4 ($\neg B_{1,1}$)
R_{10}	$\mathbf{\neg P_{1,2} \wedge \neg P_{2,1}}$	De Morgan on R_9 ⇒ neither [1,2] nor [2,1] has a pit ✔

Finding a proof can be framed as a **search problem**: initial state = the KB; actions = applying inference rules; result = KB plus the new sentence; goal = a state containing the sentence to prove. It can be much faster than model checking because irrelevant symbols ($B_{2,1}$, $P_{1,1}$, $P_{2,2}$, $P_{3,1}$) are simply ignored.

Monotonicity: the set of entailed sentences can only *increase* as information is added: if $\text{KB} \models \alpha$ then $\text{KB} \wedge \beta \models \alpha$. New knowledge can help draw more conclusions but cannot invalidate a conclusion already drawn.

4.6 Proof by resolution ★ Very likely

The rules above are sound, but are they *complete*? **Resolution** is a single inference rule that gives a complete inference algorithm when coupled with any complete search. Continue the Wumpus story: the agent returns from [2,1] to [1,1] and goes to [1,2], where it perceives a stench but no breeze. We add R_{11}: $\neg B_{1,2}$ and R_{12}: $B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$. As before we derive R_{13}: $\neg P_{2,2}$ and R_{14}: $\neg P_{1,3}$, and from R_3 and R_5 (biconditional elimination + Modus Ponens) R_{15}: $P_{1,1} \vee P_{2,2} \vee P_{3,1}$. Now:

R_{15}: $P_{1,1} \vee P_{2,2} \vee P_{3,1}$ with R_{13}: $\neg P_{2,2}$ ⇒ R_{16}: $\mathbf{P_{1,1} \vee P_{3,1}}$ (resolve away $P_{2,2}$)

R_{16}: $P_{1,1} \vee P_{3,1}$ with R_1: $\neg P_{1,1}$ ⇒ R_{17}: $\mathbf{P_{3,1}}$ the pit is in [3,1]!

Unit resolution

$$\frac{\ell_1 \vee \ldots \vee \ell_k, \quad m}{\ell_1 \vee \ldots \vee \ell_{i-1} \vee \ell_{i+1} \vee \ldots \vee \ell_k}$$

ℓ_i and m are **complementary literals** (one is the negation of the other). A clause + a single literal (unit clause).

Full resolution

$$\frac{\ell_1 \vee \ldots \vee \ell_k, \quad m_1 \vee \ldots \vee m_n}{\text{all literals of both except } \ell_i \text{ and } m_j}$$

where ℓ_i and m_j are complementary. E.g. $(P_{1,1} \vee P_{3,1})$, $(\neg P_{1,1} \vee \neg P_{2,2}) \Rightarrow (P_{3,1} \vee \neg P_{2,2})$.

Factoring: the resulting clause must contain only **one copy of each literal**; e.g. resolving $(A \vee B)$ with $(A \vee \neg B)$ gives $(A \vee A)$, which is reduced to just A. **Intuition:** if ℓ_i is true then m_j is false, so the rest of the second clause must be true; if ℓ_i is false then the rest of the first clause must be true. Either way the resolvent is true — resolution is sound.

COMMON MISTAKE

Resolve on **only one pair** of complementary literals at a time. From $(P \vee Q)$ and $(\neg P \vee \neg Q)$ you may **not** derive the empty clause; the correct resolvents are $(Q \vee \neg Q)$ or $(P \vee \neg P)$, which are tautologies (useless).

Conjunctive normal form (CNF)

Resolution works only on **clauses** (disjunctions of literals). A sentence is in **CNF** if it is a conjunction of clauses (an AND of ORs), e.g. $(A \vee \neg B) \wedge (B \vee C \vee \neg D)$. (k-CNF: exactly k literals per clause.) **Every propositional sentence is logically equivalent to a CNF sentence**, obtained in four steps:

WORKED EXAMPLE — CONVERT R_2: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ TO CNF

Step	Result
1. **Eliminate ⇔**: replace $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$	$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. **Eliminate ⇒**: replace $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$	$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
3. **Move ¬ inwards** (double negation, De Morgan) so ¬ appears only before literals	$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
4. **Distribute ∨ over ∧**	$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

The result is a conjunction of three clauses — ready for resolution.

MEMORY TRICK — CNF STEPS

"BIND" → **B**iconditional out, **I**mplication out, **N**egation in, **D**istribute ∨ over ∧.

The resolution algorithm (refutation)

```
function PL-RESOLUTION(KB, α) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          α, the query, a sentence in propositional logic
  clauses ← the set of clauses in the CNF representation of KB ∧ ¬α
  new ← { }
  loop do
     for each pair of clauses C_i, C_j in clauses do
        resolvents ← PL-RESOLVE(C_i, C_j)
        if resolvents contains the empty clause then return true
        new ← new ∪ resolvents
     if new ⊆ clauses then return false
     clauses ← clauses ∪ new
```

1. To show KB ⊨ α, show (KB ∧ ¬α) is **unsatisfiable**: convert KB ∧ ¬α to CNF.
2. Repeatedly resolve pairs of clauses that contain complementary literals and add each new clause.
3. Stop when either (a) the **empty clause** { } is produced — a contradiction, so KB ⊨ α; or (b) no new clauses can be added — KB does not entail α.

The empty clause (a disjunction of no disjuncts) is equivalent to *False*; it arises only from resolving two complementary unit clauses such as *P* and ¬*P*.

WORKED EXAMPLE — PROVE $\neg P_{1,2}$ FROM KB = $R_2 \wedge R_4$ BY RESOLUTION

Clauses of KB ∧ ¬α: (1) $\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$ (2) $\neg P_{1,2} \vee B_{1,1}$ (3) $\neg P_{2,1} \vee B_{1,1}$ [from R_2] (4) $\neg B_{1,1}$ [R_4] (5) **$P_{1,2}$** [negation of the goal $\neg P_{1,2}$].

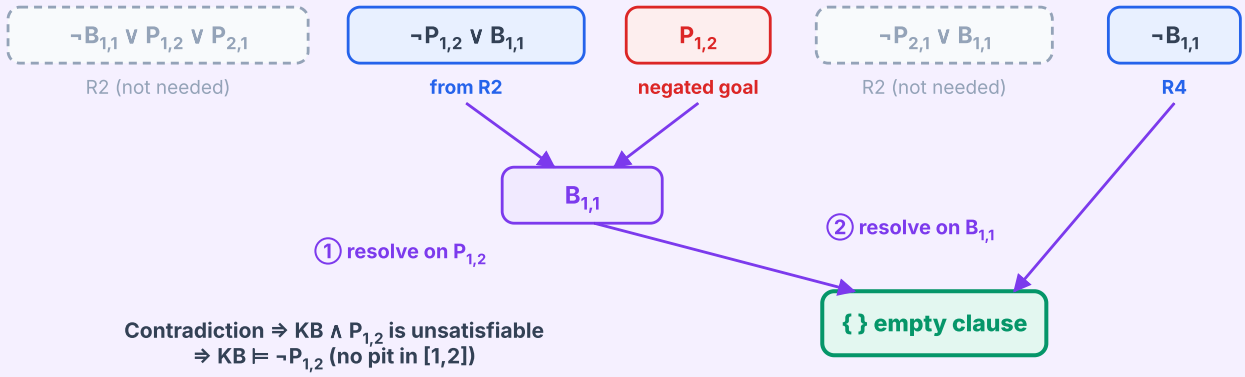


Fig 4.4 Resolution refutation tree: (2) + (5) give $B_{1,1}$; with (4) this gives the empty clause.

PRACTICE — KB: P ⇒ Q, Q ⇒ R, P. PROVE R BY RESOLUTION.

CNF clauses: (1) $\neg P \vee Q$ (2) $\neg Q \vee R$ (3) P (4) $\neg R$ (negated goal)
(1) + (3) on $P \Rightarrow$ (5) Q · (2) + (5) on $Q \Rightarrow$ (6) R · (6) + (4) on $R \Rightarrow$ **{ } empty clause** ⇒ KB ⊨ R

Completeness of resolution. The **resolution closure** RC(S) is the set of all clauses derivable by repeated resolution from the clauses S; it is finite. The **ground resolution theorem**: if a set of clauses is unsatisfiable, then its resolution closure contains the empty clause. Hence resolution is **refutation-complete**: it can always prove KB ⊨ α by refuting KB ∧ ¬α.

4.7 Horn clauses and definite clauses

Full resolution can be expensive. Many real KBs use restricted clause forms that allow very fast inference.

Clause type	Definition	Example
Definite clause	a disjunction of literals with **exactly one positive** literal	$\neg L_{1,1} \vee \neg Breeze \vee B_{1,1}$
Horn clause	a disjunction of literals with **at most one positive** literal (all definite clauses are Horn)	$\neg A \vee \neg B \vee C$; $\neg A \vee \neg B$
Goal clause	a clause with **no positive** literal	$\neg A \vee \neg B$
Not Horn	two or more positive literals	$\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$

Why they are useful:

1. Every definite clause can be written as an **implication** whose premise is a conjunction of positive literals and whose conclusion is a single positive literal: $\neg L_{1,1} \vee \neg Breeze \vee B_{1,1} \equiv (L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$. The premise is the **body**, the conclusion the **head**. A single positive literal (e.g. $L_{1,1}$) is a **fact**. Such rules are easy for humans to read.
2. Inference can be done with **forward chaining** and **backward chaining**, which are natural and easy to follow.

3. Deciding entailment with Horn clauses takes time linear in the size of the KB. (Horn clauses are also closed under resolution.)

4.8 Forward and backward chaining ★ Very likely

Running example (all definite clauses) — prove the query ***Q***:

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$

In the **AND–OR graph** (right), arrows go from premises to conclusions. An ∧ node needs *all* its incoming arrows proved (AND); several rules into the same symbol are alternatives (OR).

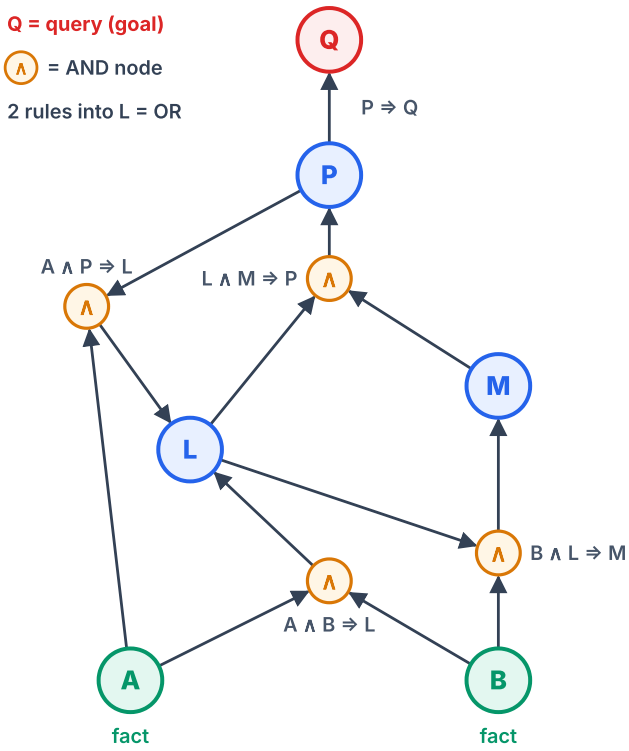


Fig 4.5 AND–OR graph of the Horn KB.

Forward chaining (data-driven)

Idea: start from the **known facts**; if all premises of an implication are known, **fire** the rule and add its conclusion to the known facts; repeat until the query is added or nothing new can be inferred. "What can I conclude from what I know?"

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
          q, the query, a proposition symbol
  count    ← a table, where count[c] is the number of symbols in c's premise
  inferred ← a table, where inferred[s] is initially false for all symbols
  agenda   ← a queue of symbols, initially symbols known to be true in KB
  while agenda is not empty do
     p ← POP(agenda)
     if p = q then return true
     if inferred[p] = false then
        inferred[p] ← true
        for each clause c in KB where p is in c.PREMISE do
           decrement count[c]
           if count[c] = 0 then add c.CONCLUSION to agenda
  return false
```

Step	Pop	Effect on rule counts (number of premises still unknown)	Agenda after
0	—	Initial counts: P⇒Q: 1, L∧M⇒P: 2, B∧L⇒M: 2, A∧P⇒L: 2, A∧B⇒L: 2	[A, B]
1	A	A∧P⇒L: 2→1; A∧B⇒L: 2→1	[B]
2	B	B∧L⇒M: 2→1; A∧B⇒L: 1→**0 ⇒ add L**	[L]
3	L	L∧M⇒P: 2→1; B∧L⇒M: 1→**0 ⇒ add M**	[M]
4	M	L∧M⇒P: 1→**0 ⇒ add P**	[P]
5	P	P⇒Q: 1→**0 ⇒ add Q**; A∧P⇒L: 1→0 ⇒ add L (already inferred)	[Q, L]
6	Q	**Q = query ⇒ return true ✔** Facts inferred in order: A, B, L, M, P, Q.	

Forward chaining is **sound** and **complete** for definite-clause KBs and runs in **linear time**. Run without a query, it stops at a **fixed point** where no new inferences are possible.

Backward chaining (goal-driven)

Idea: work backward from the **query** *q*. If *q* is known to be true, done. Otherwise find the implications in the KB whose **conclusion is *q***; if all the premises of one of them can be proved true (by backward chaining, recursively), then *q* is true. "What do I need in order to prove my goal?"

```
Goal Q?   rule P ⇒ Q          → need P
  Goal P?   rule L ∧ M ⇒ P      → need L and M
    Goal L?   try A ∧ P ⇒ L    : A is a fact ✔; P? is already on the goal stack → LOOP,
abandon this rule
              try A ∧ B ⇒ L    : A is a fact ✔, B is a fact ✔   ⇒  L proved ✔
    Goal M?   rule B ∧ L ⇒ M    : B is a fact ✔, L already proved ✔  ⇒  M proved ✔
  ⇒ P proved ✔  (L ✔ and M ✔)
⇒ Q proved ✔
```

Two checks keep it efficient: avoid **loops** (is the new subgoal already on the goal stack?) and avoid **repeated work** (has this subgoal already been proved true or failed?). Backward chaining touches only relevant facts, so its cost is often **much less than linear** in the size of the KB.

	Forward chaining	Backward chaining
Direction	facts → conclusions (bottom-up)	query → supporting facts (top-down)
Type of reasoning	**data-driven**	**goal-driven** (goal-directed)
Starts from	known facts (agenda)	the query (goal stack)
Work done	may derive many facts irrelevant to the goal	only facts relevant to the goal
Complexity	linear in the size of the KB	often much less than linear
Good for	monitoring, perception, reacting to new data ("the light turned red → what follows?")	answering specific questions ("Where are my keys?", "What shall I do now?")
Soundness / completeness	both sound and complete for definite (Horn) clauses	

EXAM TIP — FC/BC QUESTIONS (10M)

- Write the KB, draw the **AND–OR graph**, then give the FC trace as a table (pop, counts, agenda) and the BC trace as an indented goal tree.
- Mention the words **data-driven** (FC) and **goal-driven** (BC), **linear time**, and **sound & complete for Horn/definite clauses**.
- If asked "Why Horn clauses?", give the three reasons (implication form, FC/BC, linear time).

4.9 Agents based on propositional logic

The current state of the world

The agent's KB holds (i) the **percept history** and (ii) general rules about the world (its "physics"): e.g. $\neg P_{1,1}$, $\neg W_{1,1}$, a breeze rule $B_{x,y} \Leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y})$ for each square, a similar stench rule, "there is at least one wumpus" $W_{1,1} \vee W_{1,2} \vee \ldots \vee W_{4,4}$, and "at most one wumpus" $(\neg W_{x,y} \vee \neg W_{u,v}$ for every pair of squares).

Kind of variable	Meaning	Examples
Atemporal variables	aspects of the world that **never change** — no time superscript	$P_{3,1}$ (pit), $W_{1,3}$ (wumpus), $B_{2,1}$ (square is breezy)
Fluents (state variables)	aspects that **change over time** — indexed by time step t	$L^t_{x,y}$ (agent at $[x,y]$ at time t), $FacingEast^t$, $HaveArrow^t$, $WumpusAlive^t$
Percept variables	what is perceived at time t	$Stench^t$, $Breeze^t$, $Glitter^t$

Percepts are linked to atemporal properties of squares: $L^t_{x,y} \Rightarrow (Breeze^t \Leftrightarrow B_{x,y})$ and $L^t_{x,y} \Rightarrow (Stench^t \Leftrightarrow S_{x,y})$. A square is safe: $OK^t_{x,y} \Leftrightarrow \neg P_{x,y} \wedge \neg(W_{x,y} \wedge WumpusAlive^t)$.

Effect axioms and the frame problem

An **effect axiom** describes the outcome of an action at a time step, e.g.

$$L^0_{1,1} \wedge FacingEast^0 \wedge Forward^0 \Rightarrow (L^1_{2,1} \wedge \neg L^1_{1,1})$$

Effect axioms say what **changes**, but not what **stays the same**. After *Forward* at time 0, can the agent prove $HaveArrow^1$? **No!** Nothing says the arrow is still there. This is the **frame problem**. One fix is **frame axioms** stating what does not change, e.g. $Forward^t \Rightarrow (HaveArrow^t \Leftrightarrow HaveArrow^{t+1})$, but with m actions and n fluents we need $O(mn)$ of them (the *representational* frame problem).

SUCCESSOR-STATE AXIOM (THE SOLUTION)

Write one axiom per fluent F, describing its value at $t{+}1$ in terms of the actions that can change it:

$$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t)$$

In words: F is true next time iff some action just made it true, or it was already true and no action made it false. Example: $HaveArrow^{t+1} \Leftrightarrow (HaveArrow^t \wedge \neg Shoot^t)$. For location: $L^{t+1}_{1,1} \Leftrightarrow (L^t_{1,1} \wedge (\neg Forward^t \vee Bump^{t+1})) \vee (L^t_{1,2} \wedge FacingSouth^t \wedge Forward^t) \vee (L^t_{2,1} \wedge FacingWest^t \wedge Forward^t)$.

A hybrid agent

The **hybrid Wumpus agent** combines **logical inference** (to work out which squares are safe, where the wumpus may be, etc.) with **search** (A* to plan routes through safe squares) and condition–action style priorities.

```
function HYBRID-WUMPUS-AGENT(percept) returns an action
  inputs: percept, a list, [stench, breeze, glitter, bump, scream]
  persistent: KB, initially the atemporal "wumpus physics"
              t, a counter, initially 0;  plan, an action sequence, initially empty
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  TELL the KB the temporal "physics" sentences for time t
  safe ← {[x,y] : ASK(KB, OK^t_{x,y}) = true}
  if ASK(KB, Glitter^t) = true then
     plan ← [Grab] + PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
  if plan is empty then
     unvisited ← {[x,y] : ASK(KB, L^{t'}_{x,y}) = false for all t' ≤ t}
     plan ← PLAN-ROUTE(current, unvisited ∩ safe, safe)
  if plan is empty and ASK(KB, HaveArrow^t) = true then
     possible_wumpus ← {[x,y] : ASK(KB, ¬W_{x,y}) = false}
     plan ← PLAN-SHOT(current, possible_wumpus, safe)
  if plan is empty then          // no choice but to take a risk
     not_unsafe ← {[x,y] : ASK(KB, ¬OK^t_{x,y}) = false}
     plan ← PLAN-ROUTE(current, unvisited ∩ not_unsafe, safe)
  if plan is empty then
     plan ← PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
  action ← POP(plan)
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action

function PLAN-ROUTE(current, goals, allowed) returns an action sequence
  problem ← ROUTE-PROBLEM(current, goals, allowed)
  return A*-GRAPH-SEARCH(problem)
```

Priorities in plain words: (1) if there is a glitter, grab the gold, go home and climb out; (2) otherwise visit the nearest **safe unvisited** square; (3) if none, try to **shoot** the wumpus if its possible location is known; (4) if still nothing, take a **risk** on a square that is not provably unsafe; (5) if the mission is impossible, go back to [1,1] and climb out.

Logical state estimation (brief)

Problem: ASK has to reason over the **whole percept history**, so inference gets slower as t grows. Solution: keep a **belief state** — a representation of the set of all possible current states of the world — as a logical formula over the current fluents, and update it after every action and percept (**state estimation**). Exact belief states can be exponentially large, so agents often use a **conservative approximation** (e.g. a conjunction of literals, 1-CNF: keep only the facts known for certain) and cache inference results.

Making plans by propositional inference (SATPlan, brief)

Instead of A*, plans can be found with a SAT solver. Build one sentence: $Init^0 \wedge Transition^1 \wedge \ldots \wedge Transition^T \wedge Goal^T$ (initial state, successor-state axioms for each step, goal true at time T). If a SAT solver finds a model, the action symbols that are *true* in it form the plan. Try T = 0, 1, 2, ... up to T_{max}. Extra axioms are needed: **precondition axioms** (e.g. $Shoot^t \Rightarrow HaveArrow^t$) and **action exclusion axioms** ($\neg A^t_i \vee \neg A^t_j$: only one action at a time).

```
function SATPLAN(init, transition, goal, T_max) returns solution or failure
  for t = 0 to T_max do
     cnf   ← TRANSLATE-TO-SAT(init, transition, goal, t)
     model ← SAT-SOLVER(cnf)
     if model is not null then return EXTRACT-SOLUTION(model)
  return failure
```

EXAM TIP — LOGIC QUESTIONS

- **Wumpus world (10M):** draw the 4×4 grid with START, pits, wumpus, gold, breezes and stenches; then PEAS; then environment properties; then the step-by-step reasoning with small grids.
- **Entailment/soundness/completeness (6M):** definitions with symbols $\models$ and $\vdash$, $M(KB) \subseteq M(\alpha)$, the 8-model table and the Venn figure.
- **Resolution (10M):** write the clauses of $KB \wedge \neg\alpha$ in CNF (show the 4 steps), then draw the resolution tree ending in the empty clause, and conclude "therefore $KB \models \alpha$".
- Write subscripts carefully ($P_{1,2} \neq P_{2,1}$) and name the rule at every step — that is where step marks are given.

COMMON MISTAKES

- $P \Rightarrow Q$ is **true** when P is false; it is false only for P = true, Q = false.
- Writing $\vdash$ (derivation) where $\models$ (entailment) is meant, or swapping the definitions of **sound** and **complete**.
- In resolution, forgetting to **add the negated goal**, or forgetting to convert to CNF first.
- Horn clause = **at most one** positive literal; definite = **exactly one**.
- FC goes from facts to the goal; BC goes from the goal to facts — do not reverse them.

LIKELY EXAM QUESTIONS — CHAPTER 4

1. What is a knowledge-based agent? Write the generic KB-AGENT program and explain TELL and ASK. Distinguish declarative and procedural approaches. **(6M)**
2. Describe the Wumpus world with its PEAS and environment properties. Explain how the agent reasons in the first few moves. **(10M)**
3. Define syntax, semantics, model, entailment, logical inference, soundness, completeness and grounding. Illustrate entailment with the Wumpus example. **(8M)**
4. Give the syntax (BNF) and semantics (truth table) of propositional logic. **(6M)**
5. List the standard logical equivalences. Define validity, satisfiability and the deduction theorem. **(6M)**
6. Using Modus Ponens, And-Elimination and equivalences, prove $\neg P_{1,2}$ from R_1–R_5. **(6M)**
7. Convert $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ to CNF and prove $\neg P_{1,2}$ using resolution. **(10M)**
8. What are Horn and definite clauses? Explain forward and backward chaining with the AND–OR graph example. Compare them. **(10M)**
9. Explain fluents, effect axioms, the frame problem and successor-state axioms. **(6M)**
10. Explain the hybrid Wumpus agent. **(5M)**

60-SECOND RECAP

- KB = set of sentences; agent TELLs percepts, ASKs for actions; inference derives new sentences.
- Wumpus PEAS: +1000 gold out, −1000 death, −1 per action, −10 arrow; actuators Forward/TurnLeft/TurnRight/Grab/Shoot/Climb; percept [Stench, Breeze, Glitter, Bump, Scream].
- KB ⊨ α iff M(KB) ⊆ M(α); sound = derives only entailed sentences; complete = derives all entailed sentences.
- Connectives ¬ ∧ ∨ ⇒ ⇔ (that precedence); $P \Rightarrow Q$ false only for T ⇒ F. Model checking is $O(2^n)$.
- α ⊨ β iff α ∧ ¬β unsatisfiable. Rules: Modus Ponens, And-Elimination, equivalences. CNF steps: BIND. Resolution + refutation is complete.
- Horn: ≤ 1 positive literal; FC (data-driven) and BC (goal-driven) are linear-time, sound and complete for definite clauses.
- Fluents change with time; frame problem solved by successor-state axioms; hybrid agent = logic for safety + A* for routes.

UNIT I — LAST-MINUTE FACT SHEET

Rational agent	maximizes *expected* performance given percept sequence + built-in knowledge
PEAS	Performance, Environment, Actuators, Sensors
Environment dimensions	observable, agents, deterministic, episodic, static, discrete, known
Agent types	simple reflex → model-based → goal-based → utility-based → learning
Problem components	initial state, actions, transition model, goal test, path cost
BFS / UCS / DFS	FIFO, $O(b^d)$ / priority by g, $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$ / LIFO, $O(b^m)$ time, $O(bm)$ space
DLS / IDS / Bidirectional	$O(b^l)$, $O(bl)$ / $O(b^d)$, $O(bd)$ / $O(b^{d/2})$
Greedy vs A*	$f = h$ (450 km) vs $f = g + h$ (418 km, optimal)
Admissible / consistent	$h(n) \le h^*(n)$ / $h(n) \le c(n,a,n') + h(n')$
8-puzzle example	$h_1 = 8$, $h_2 = 18$, true cost 26; 9!/2 = 181,440 states
Effective branching factor	$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$
Entailment	KB ⊨ α iff M(KB) ⊆ M(α) iff KB ∧ ¬α unsatisfiable
CNF of R_2	$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$
Successor-state axiom	$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t)$